SCHOOL OF COMPUTER SCIENCE

UNIVERSITY OF NOTTINGHAM

Automatic Music Transcription for Guitar: A Machine Learning Approach to Creating a Guitar Practice Tool

Dissertation

Student ID:
20547604

Supervisor:
Juan Martinez Avila

Course:
COMP3003

Program:
BSc Hons Computer Science with Artificial Intelligence

I hereby declare that this dissertation is all my own work, except as indicated in the text.
AS

April 14, 2026

Abstract

Guitar Music Transcription (GMT) is the challenge of converting raw polyphonic audio into guitar tablature, which is used by musicians to learn and play music on the guitar. This research investigates how GMT can be implemented and used as a tool to help improve users' ear training, the process of developing the ability to identify notes by ear and map them to specific fretboard positions. Addressing the technical challenge of pitch redundancy in guitar transcription and the education gap where existing GMTs are not designed for interactive ear training.

The methodology evaluates three distinct approaches: a simple heuristic baseline, a sequential 1D Convolutional Neural Network (CNN) leveraging Spotify's Basic Pitch for signal processing, and an end-to-end 2D CNN. The models were evaluated using a dataset to compare the performance of the models against each other comparing the prediction accuracy of the model to the actual note that was played. The dissertation focuses on creating a complete project for the user, this includes developing a Graphical User Interface and features such as MIDI player and "traffic light" confidence scoring system to guide user corrections.

The results of the tests showed the sequential model pipeline performed better than the end-to-end model. The sequential pipeline achieved an F1-score of 0.833 for pitch detection using Basic Pitch with post-processing, and a fretboard exact-match accuracy of 60.3% on the GuitarSet dataset, outperforming the heuristic baseline (49.6%) by over 10 percentage points. The end-to-end GOAT CNN achieved 17.4% exact-match accuracy, limited by onset detection capturing only 25.6% of ground-truth notes. The sequential CNN performed best when the model included a heuristic fallback to choose the closest string/fret to the previous note. The system is lightweight, operates locally to protect copyright, and integrates interactive features including a confidence-based traffic light system and MIDI playback comparison to support an ear-training workflow. To our knowledge this is the first research using a GMT system aimed specifically at interactive ear-training.

Index Terms: Guitar Music Transcription, Polyphonic Audio, Fretboard Mapping, Convolutional Neural Network, Tablature

Contents

1 Introduction

1.1 Background and Motivation

The availability of guitar tablature (musical notations for stringed instruments such as the guitar) for a specific composition is highly dependent on the song's popularity. If there is no tab available for the song, the only alternatives are to learn the song by ear, which is challenging and requires either a high level of expertise or paying a professional transcriber. Manually created tabs also come under substantial scrutiny regarding their accuracy, as users may publish unverified or inaccurate tabs [1]. The inaccuracy can be a significant barrier for entry-level self-taught learners, leading to frustration and hindering their progress [2]. This research proposes the development of an intelligent guitar practice tool using automatic music transcription to address challenges users face.

An Automatic Music Transcription (AMT) is a tool used to convert a music signal into musical notation that can be read by musicians to play their instrument [3]. In the context of this study, the goal is to convert guitar play into guitar tablature, i.e. music notation for documenting frets and string instruments. Building an AMT is a challenging task that includes multiple sub-tasks that need to be handled, including: Onset Detection, Pitch Detection, Note Tracking, Handling polyphonic audio [4], Effects and various guitar techniques [5]. Leading transcription systems are still significantly below the level of a human expert [3].

While AMT may not match a music expert, the progress it has made suggests it can be a viable alternative to the problem and can still be used by users to correct any inconsistencies in the tab generated or be used as an assistive educational tool to allow users to confirm if the tab that they created is correct. As an intelligent guitar practice tool, it will include segmentation of tab generation to practice sections of the song, improving users' development in learning a tab. The option of exporting to industry-standard format gives users the option to use their preferred tool [6].

1.2 Aims and Objectives

The primary aim of this research is to be able to design and evaluate an assistive practice system for guitarists, by integrating AI and pitch detection with features including fretboard mapping to create a functional aid. To achieve this, an end-to-end system and a sequential transcription pipeline will be tested. Comparing these frameworks will determine which methodology would best balance pitch accuracy with the practical educational utility required for practice, specifically optimising ergonomic playability of the generated tabs.

In order for this project to be deemed successful, the main objectives have been established as follows:

1. Research previous AMT approaches (Fretting Transformer [7], TabCNN [8]) and benchmark the models used to compare effectiveness of pitch detection accuracy, fretboard mapping performing using a dataset.

2. Develop a sequential process pipeline: 1. pitch detection model using Basic Pitch [4] to convert audio to MIDI, 2. Various fretboard mapping models namely simple heuristic, CNN context based solution.

3. Develop an end-to-end process, processes raw audio and generates a string and fret output.

4. Develop a software-based user interface, implementing options to modify tabs generated, export, slicing sections and converting tab generation to be exported to industry-standard format (MusicXML [9], .gp5 Guitar Pro, PDF) to enable compatibility with existing guitar tab software (e.g. MuseScore, Guitar Pro)

5. Generate guitar technique classification (bends, pull-offs, etc) from audio feature. Training the model using SynthTab's [10] dataset and GOAT dataset [11] evaluating novel features of the tool.

6. Evaluating the system's performance (monophonic audio - single guitar note, polyphonic audio - multiple guitar notes, multi-instrument polyphony audio - mix band audio), Measuring model evaluation metrics using F1-score, Recall, Precision, string/fret accuracy, and technique detection accuracy. Comparing findings with other models (TabCNN [8] and heuristic baseline). Targeting to achieve a F1 score over 0.8.

1.3 Requirements Analysis

To systematically define the scope of the intelligent practice tool, the requirements were prioritised using the MoSCoW method (Must have, Should have, Could have, Won't have). This ensures the core functionality of the Automatic Music Transcription (AMT) system is established before developing additional features.

Priority	Functional Requirements	Non-Functional Requirements
Must Have	- Process polyphonic audio (.wav, .mp3) - Generate ASCII tablature from audio - Provide a GUI to display and edit tabs - Export to PDF and GuitarPro5 (GP5)	- Inference must run locally to protect user copyright - The GUI must remain responsive during model execution - Lightweight enough to run on consumer hardware
Should Have	- Confidence scoring for predicted notes - Fallback heuristic for fretboard mapping	- High precision to avoid confusing beginner users
Could Have	- Audio isolation (Demucs) to separate backing tracks - Tempo detection and automatic bar grouping	- Modular architecture to easily swap deep learning models
Won't Have	- Real-time live audio transcription - Automatic technique detection (bends, slides)	- Server side processing

Table 1: MoSCoW Requirements Analysis for the AMT Practice Tool

2 Related Work

An Automatic Music Transcription (AMT) tool has been a concept that has remained a subject of significant interest for researchers and musicians for a long time. Guitar Music Transcription (GMT) is a subsection of AMT that focuses on the specific challenges of transcribing audio to guitar tablature. Complete solutions comparable to the proposed system exist, such as Songsterr AI tab generation. These show the advanced leaps in progress that have been made in this field. Due to it not being feasible to compare the proprietary system, the model will be benchmarked against open-source AMT research solutions. The development in research of AMT has shifted from building heuristics (e.g A* search) to implementing deep learning to create sophisticated models to handle the nuances and challenges of music processing. Due to the nature of polyphonic audio and challenges such as pitch redundancy, multiple approaches to this problem have been made. One of the methods

that has been explored includes using hexaphonic pickups to generate transcriptions by analysing signals in each individual string.

2.1 Music Education Technology & Guitar Practice Tool

There is plenty of research on the benefit of ear training for guitarists, which develops pitch recognition and overall musical comprehension. By gaining these skills guitarists would be able to accelerate their ability to transcribe music and play by ear [12]. Guitarists would be able to naturally navigate the fretboard.

Generally, these skills are taught by a mentor tutoring through drills which often fails to account for individual learning paces or provide immediate personalised feedback and also needs to take the cost into consideration. Another option is to use books and sheets or unverified tablature. However, this can lead to guitarists learning passively which leads to guitarists relying on them without connecting what they hear to what they play.

There exists software and applications designed for tackling these challenges, by offering adaptive learning paths and real time feedback. These digital tools bridge the gap between formal training and independent learning, enabling users to engage in ear training where they can receive feedback and error correction [13]. This offers a way of connecting music theory to real music context, providing a faster way to learn. Some platforms such as Yousician and SmartMusic use AI algorithms which create a personalised path adapting to users progress. A key limitation of Yousician and SmartMusic is that they are based on their curated music library, users can only practice and engage based on their pre-loaded content. Therefore cannot be used to transcribe and verify a user's own musical material.

The key factor to learning is through feedback in music education research, identifying errors in them and correcting them enables learners to develop [14], the tool establishes a system with uncertainty that requires corrections rather than a perfect tablature, with features like confidence-based traffic light system this allows for learners to put their attention to notes that most likely require correction.

While technical transcription research has advanced considerably, no existing system has been designed to implement it into an ear practice tool system by specifically following a workflow: listening, predicting, comparing, and correcting. This dissertation aims to address the gap present by building a system that tackles the challenges discovered.

2.2 Pitch Estimation Methods

A popular option is to use a probabilistic approach like the pYin algorithm [15], which is an extension of the Yin algorithm [16] to solve the limitation during post-processing (cleaning raw audio) where it cannot fall back on alternative interpretations of the signals. pYin overcomes the problem by outputting multiple pitch candidates with associated probabilities, reducing the loss of useful information. The core reason why pYin would not be an ideal option is due to being a monophonic transcription tool, while an AMT would require a multi-instrument polyphony transcription. A deep learning solution includes CREPE [17], designed for single-f0 estimation and effectively fails to transcribe polyphonic audio.

Because Basic Pitch is lightweight, it can operate in low-resource settings, making the tool accessible to a much wider range of users. While being lightweight, it still outperforms other strong instrument-agnostic baseline models across nearly all datasets. Basic Pitch performance can be impacted when tested with a full band mix (multiple instruments playing, i.e, songs), as it was designed for polyphonic recordings of a single instrument class. Basic Pitch begins by converting the audio input to a Constant-Q Transform (CQT). Using a technique called harmonic stacking, it is able to capture harmonic-related information. This works by copying and vertically shifting the CQT representation by the number of frequency bins corresponding to each harmonic. The model is trained using binary cross-entropy of the three outputs (pitch, noise, onset). Therefore, improving the note accuracy.

2.3 Fretboard Mapping Approaches

Fretting transformer [7] is a pitch-to-tab process to convert MIDI to guitar tablature, tackling complex issues such as pitch redundancy. It uses an encoder-decoder model utilising the T5 transformer architecture (Text-to-Text Transfer Transformer) [18] to perform tasks including sequence-to-sequence mapping (MIDI note to guitar tablature), fingering prediction, while ensuring context is taken into account.

2.4 End-to-End Vs Sequential Pipelines

There are two strategies for creating an AMT. This includes an end-to-end system or the sequential pipelines.

An end-to-end system such as TabCNN [8]. Rather than the sequential options explored previously, an end-to-end approach attempts to solve both the steps, including audio to MIDI and MIDI to tablature, by transcribing directly into tablature without an intermediate step like MIDI conversion. From processing the audio spectrogram, they are able to access more music

context, theoretically allowing them to distinguish between the same pitch played on different strings based on timbre differences, solving the problem of pitch redundancy. TabCNN is a convolutional neural network (CNN) designed to estimate individual strings/frets from the audio. TabCNN performed well on the GuitarSet dataset, achieving a Tablature Disambiguation Rate over 89%, meaning 89% of detected pitches were assigned to the correct fingering. Since it was trained on the GuitarSet, TabCNN does not account for various guitar techniques, which would be a critical flaw when providing it as a tool for users. The author also mentioned adding temporal smoothing to improve performance as future work. The recent architecture, like MT3 [19] that has been proposed, includes using transformers for its multi-transcription functionality, advancing end-to-end systems' capabilities.

Technique Aware Audio to Tab Representation Tool (TART) [20] is a hybrid attempt to bridge the gap by including four stages to transcribe audio to tab. The model's goal is classify guitar techniques and handle the case of pitch redundancy. The stages include: 1. fine-tuning a piano transcription model which uses a Convolutional Recurrent Neural Network (CRNN) to train on a guitar dataset, 2. Use an MLP-based classifier to extract feature vectors from the audio and update the note events with a technique label. 3. Using a transformer architecture inspired by the fretting-transformer approach to deal with the pitch redundancy confusion to predict the string and fret combination for each note. 4. Merge the data information of the 3 processes to create an ASCII guitar tablature. One of the limitations encountered by TART was the issue of overfitting to the training model, and this was seen when training the model with the Magcii guitar dataset and testing on another (IDMT) dataset. One approach to improve the model would be to train the model on a significantly larger dataset and to diversify the data.

2.5 Technique Classification

Guitar techniques play a major role in achieving realistic tablature. Expression styles executed with the fretting hand, such as bends, slides and vibrato, and there are legato techniques like hammer-ons and pull-offs contributing to the continuity of the musical phrase. These multiple factors make guitar technique classification a continuous obstacle. These challenges include the complexities inherent in the guitar audio signal, the difficulties in annotations, and the limited availability of diverse training data requiring knowledge in music theory to be able to analyse the acoustic and timbral features of the audio. For example, to figure out expression styles, it requires analysing the inharmonicity (frequency shifts of partial vibration) to differentiate between which string a pitch was played on.

In the early stages, a heuristic approach was adopted, and later, using feature extraction paired with a classifier like Support Vector Machine (SVM) [5]. The modern approach to the problem is to use deep learning techniques such as Technique-Aware Audio-to-Tab

(TART) [20], which integrates a Multi-Layer Perceptron (MLP) classifier into its pipeline to amend note events with labels such as "bends" or "vibrato". A model that joins pitch detection and guitar technique classification is TENT [21] using a CNN architecture to perform playing technique detection. Although these models currently struggle to pick up legato techniques, it will be a challenge that needs to be tackled.

The bottleneck to the performance of these classification models is the diversity of the training data. Datasets like GuitarSet provide annotations to fret and string, but they do not label the guitar techniques. Newer datasets, including GOAT, aim to solve this problem by providing a larger dataset of annotated frets and strings, adding guitar techniques, and offering a more robust solution to train models.

2.6 Datasets

Developing a robust AMT tool is fundamentally dependent on the dataset used to train the model. One of the commonly used options includes GuitarSet [22], which contains 360 annotated acoustic guitar recordings utilising the hexaphonic pickup setup that was mentioned. Since it uses an acoustic guitar, it would not perform as well for electric guitar transcription, and the audio, being a total size of 180 minutes, can lead to issues with overfitting. SynthTab was developed to address the scarcity of annotated guitar data by using synthesised audio, but this leads to it failing to capture the subtle nuances that occur when a user plays the guitar. The large scale of the dataset remains crucial for training the models and mitigating the overfitting issue. Finally, another dataset that is worth looking at is GOAT (Guitar On Audio and Tablature), which focuses on recording electric guitar audio and comprises 5.9 hours of direct input recordings from a variety of guitars and players. Since it contains covers of popular songs, it is not publicly accessible and requires requesting access for research purposes only.

2.7 Music Segmentation

Music segmentation aims to improve practice for users by separating the tabs into their musical sections which are intro, verse, chorus and bridge. This is known as Audio-Based Music Structure Analysis (MSA) [23]. MSA methods assume musical homogeneity, and conclude boundaries as sudden changes in musical attributes and to take repetitions in musical structure into consideration. MSA can be carried out using probabilistic models such as Hidden Markov Models [24] or Matrix Factorisation techniques to simultaneously determine the precise boundaries and assign labels based on musical similarity. A modern approach uses deep learning models such as CNNs by training as binary classifier directly to the spectral representation to automatically learn and detect musical boundary locations

with high temporal accuracy by leveraging patterns found in audio features [25]. These models achieve improved performance on benchmarks like SALAMI [26]. This feature was descoped from the project due to time requirement and was less of a priority compared to the other features for the usage as an ear training tool as it would not directly help improve a users practice.

2.8 Summary & Research Gaps

Currently there are two types of sequences to building an AMT such as sequential pipeline like Fretting Transformer which achieves a high fretboard accuracy but this assumes the input provided is clean MIDI, and end-to-end systems like TabCNN that handle raw audio but require large dataset and struggles to generalise. There are also hybrid approaches like TART which can do technique classification but introduces complexity making it less suitable for lightweight systems. Generally, none of these systems are designed as interactive practice tool rather they are transcription tools to produce tabs with no mechanism to allow users to verify, correct or learn from the tabs. This dissertation addresses the gap by building the two types of models, evaluating them and providing an interactive GUI, confidence scoring, MIDI player comparison, explicitly targeting the ear training use case that existing research has left unaddressed

3 Methodology

3.1 Model Justification

To design the system, three models are going to be developed this includes, a heuristic model, sequential pipeline, and an end-to-end pipelines. The purpose of the heuristic algorithm is to perform as a baseline to compare the various machine learning models results to it and justifies the requirement of the other models. For the sequential pipeline, it was decided to use a 1-Dimensional CNN over a Long Short Term Memory (LSTM) as the model needs to focus on local pattern rather than needing to take into consideration the global context. The purpose of the model is to take in MIDI input and generate a string/fret output. A 1D CNN perfectly fits these needs rather than using a Convolutional Recurrent Neural Network (CRNN), which would have overcomplicated the system. For the end-to-end model a 2D CNN was chosen as it needs to process audio to fret/string through the handling of tensors. The 2D CNN fits the needs as it contains a context window which is an audio slice giving the model context from surrounding notes to determine the current note's string and fret positions. This reduces the need for a CRNN which would incorporate LSTM to factor in temporal memory which would allow the model to look back to consider what string/fret should be

played. A CRNN requires more parameters and a larger dataset to prevent overfitting, plus the added complexity to build this system would have been a challenge in the given time frame so this approach was not pursued.

3.2 Pitch Detection Method

Basic Pitch is a lightweight, instrument-agnostic model for AMT responsible for generating MIDI of the frequency audio through a lightweight convolutional neural network with a shallow architecture to gain data, including notes (pitch, onset, offset) and multi-pitch to be then used to find musical information. To circumvent the issue, the audio is isolated through Demucs [27]. However, this introduces a significant computational cost to generate the isolated guitar track, and this will need to be taken into account when building a tool for users. Basic Pitch was chosen over the other possible option due to its ability to transcribe polyphonic signals and based on the Basic Pitch research paper [4] it outperforms competitors including Technique-Embedded Note Tracking (TENT) [21] which is a guitar specific transcription tool where TENT achieved a F-measure (Fno) of 0.763, compared to Basic Pitch's baseline 0.840.

3.3 Fretboard Mapping Algorithms

One of the main challenges that differentiates piano transcription from guitar is the ability to play the same note in various positions of the fretboard. Fretboard mapping gives the user options with tunings, allowing users to select the type of tuning from a list, so the tab provided will be tailored. The open-string MIDI numbers is predefined for each tuning. Another feature implemented is to take into account for capo position. As the capo position increases, the pitch of each open string raises by one semitone.

Let $P = \{p_1, ..., p_n\}$ be a sequence of detected MIDI pitches and $T = \{(s, m_s)\}$ be the guitar tuning, where $s \in \{1, ..., 6\}$ is the string number and m_s is the open string MIDI pitch.

For each pitch p_i, we generate candidate positions:

$$C_i = \{(s, f) \mid f = p_i - m_s,\ 0 \leq f \leq 24\}$$

We select the best position using a greedy strategy:

$$\phi_i = \begin{cases} \arg\min_{(s,f)\in C_i} |f - 5| & i = 1 \\ \arg\min_{(s,f)\in C_i} d(\phi_{i-1}, (s, f)) & i > 1 \end{cases}$$

where $d((s_1, f_1), (s_2, f_2)) = |s_1 - s_2| + |f_1 - f_2|$ is the Manhattan distance.

The fretboard mapping uses a greedy algorithm to choose the local optimal choice as the next step, where it uses the nearest neighbour heuristic to find the next closest to the previous string and fret. The algorithm finds all possible positions using the function `map_note_to_fret` to calculate fret for each string, keeping only valid fret positions and listing the candidates. `best_position` aims to select a position close to the middle of the fretboard (5th fret), as it assume that most users play in the middle of the fretboard. If there is a previous note, it takes this into consideration and calculate the minimum distance to the next note [28]. Using pathfinding, where it treats each note as a node and transitions as edges. This results in the issue of the fretboard mapping not containing any musical context, not looking ahead, and not being aware of chord shapes. This approach does not guarantee that it can simulate how a human would play. The heuristic was chosen as a baseline so to strictly evaluate the importance of context in comparison to see how it improves accuracy. It was designed as a baseline to simulate how a beginner might navigate the fretboard by always selecting the nearest fret position.

Furthermore, both the heuristic and CNN approaches operate within a sequential pipeline mitigating the risk associated with end-to-end systems (like TabCNN). Where, unlike an end-to-end system where a failure in audio processing can result to a total failure in the transcription. The current approaches decouples the two process, to ensure even if there are errors during the pitch detection, it will still be capable of generating tablature.

3.4 Sequential Fretboard CNN Architecture

An alternative approach to mapping MIDI to the fretboard was developed using a temporal 1-D Convolutional Neural Network. A 1-D CNN was specifically chosen because music is time-series data (temporal), which means it is able to understand neighbour relationships. Compared to the simple heuristic, the CNN here gains context on the melodic patterns and position on the fretboard from training the model with the SynthTab dataset. It does this by having a context window of 5 (5 before, current, 5 after), giving an input sequence length of 11. This window size was chosen on the physical constraint of guitar hand position: a guitarist's fretting hand typically remains in one position for a short melodic phrase before shifting. Therefore, having a window of ± 5 notes is long enough to capture the current position of the hand, but still short enough to not include completely different sections of the piece. Having a longer context window would also be computationally expensive, going against the non-functional requirement of keeping the tool lightweight. The model extracts data from the SynthTab's jam files and individual string recordings to learn frets and strings patterns. The MIDI number is embedded in the vector so the model can learn the semantic relationship between notes. The current model uses 3 convolutional blocks, producing a joint classification model with 150 classes to predict the string and fret from the extracted features. It uses channel depths of 32, 64, 128, to 256 to use channel expansion as the channel size

increases to capture complex music features of the data. The rectified linear unit (ReLU) activation function applies nonlinearity to the model, and dropout is set to 0.25 to prevent overfitting, as it randomly drops nodes in the CNN during training, preventing the network from becoming too dependent on certain nodes and encouraging it to learn generalised features.

After every convolutional block, Batch Normalisation is applied to balance the convergence speed and stability. The data is then flattened from 3D tensors to a 2D tensor for the joint classifier layer. This layer contains 150 classes because it includes the combination of strings and fret positions: (6 (strings) $\times$ 25 (fret) = 150 (classes))

Both CNN output 150 joint classes rather than having the string and fret outputted separately as they are not independent variables on a guitar, to tackle the challenge of pitch redundancy having the string and fret separate leads to them being classified at the wrong different string/fret as it does not consider what the fret or string is, when making judgement for example the model would output the string from one note and the fret from the pitch redundancy of the note. By having 150 joint class the model learns valid combinations from the data.

The model parameters were optimised using the Adam optimiser (adaptive learning rate) with an initial learning rate of 0.001 for efficient convergence. Parameter updates are made from standard backpropagation. Reduce Learning Rate on Plateau (ReduceLROnPlateau) scheduler is used to lower the learning rate if the validation does not improve for 3 consecutive epochs. Lastly, to prevent overfitting and promote generalisation, the training loop monitors the lowest validation loss and utilises model check-pointing to save only the best-performing model's weights to disk. The target is converted to a single integer index and minimised using Cross Entropy Loss. The current version of the model was trained on a subset of 5,000 audio files from the SynthTab dataset over 26 epochs converging smoothly with no gap between the training and validation, this shows the model didn't fall into overfitting.

Pitch redundancy leads to the model predicting positions that are physically far apart on the fretboard. A Heuristic fallback is added to ensure that a closer note is selected rather than a note that is far apart from the previous note. This ensures the model does not randomly select a note far apart and instead will select the correct note that is close to the previous notes. To implement this function a similar heuristic to the baseline was developed to carry out accurate prediction.

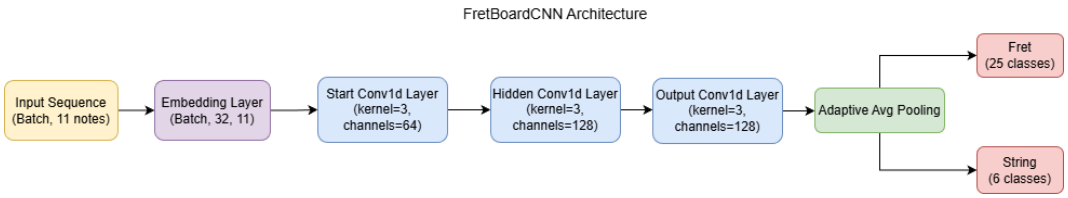

Figure 1: Process of CNN

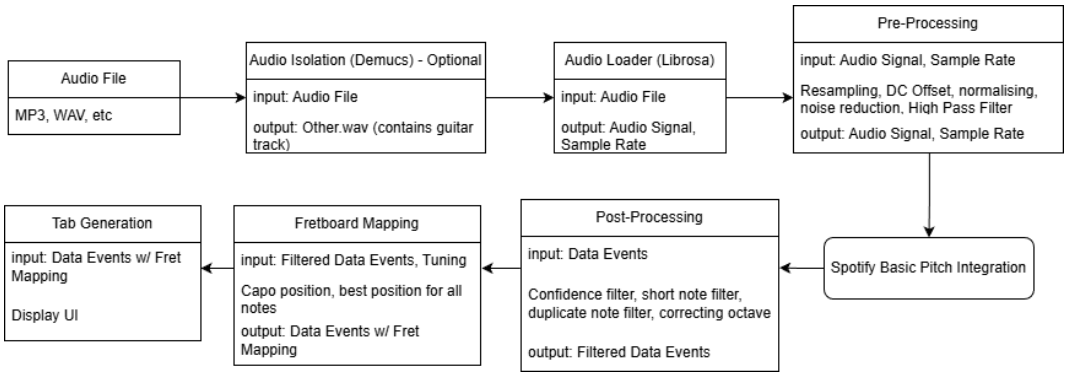

Figure 2: Process Pipeline

4 Implementation

4.1 Guitar Audio Isolation

Guitar Audio Isolation is an optional process for multi-instrument audio. It separates the guitar track from other sources such as vocals, drums, and bass. This process utilises Demucs [29], an open source audio isolation tool from the Facebook research team. The MDX Extra and MDX Extra Q models were evaluated for this purpose. Both models create isolated tracks in the form of vocals, drums, bass and others. The guitar track is stored in other.wav, but it also leads to the issue of other instruments being included as well, so the performance can vary based on the audio. The difference between the two is that the MDX extra Q is a faster model but leads to a cost in quality, while the MDX extra takes a longer time to process.

4.2 Pre-Processing Pipeline

The purpose of pre-processing the audio signal is to filter and clean the signal for accuracy when put into the pitch detection. In this case, it is through Spotify's Basic Pitch. The first line of processing is to resample the audio to set the sample rate at 22050Hz, since Basic Pitch internally resamples it before processing as it might affect the accuracy of the Basic Pitch Model. DC offset moves the audio wave to zero amplitude to prevent any premature

clipping of peaks. The audio is then normalised, which is where the audio signal amplitude is scaled so that it meets a target level. The goal of normalising the audio here is to set the audio signal to be consistent.

4.3 Post-Processing Pipeline

Once Basic Pitch returns the data events, the go through post processing to ensure that accurate tablature can be created. This is done by filtering based on a confidence threshold, ensuring that only notes for which Basic Pitch reports high confidence are retained. The next step is the short note filter that takes the duration of the note (offset-onset) and checks to see if it crosses the required duration threshold. The notes are then sorted in ascending order based on the onset time so that the correct order of notes is on the tabs. Duplicate notes are filtered based on whether the notes duplicate before the duration threshold. Finally, it goes through octave error correction, which is to match the octave to the constraints of the guitar range.

4.4 End-to-End GOAT CNN Pipeline

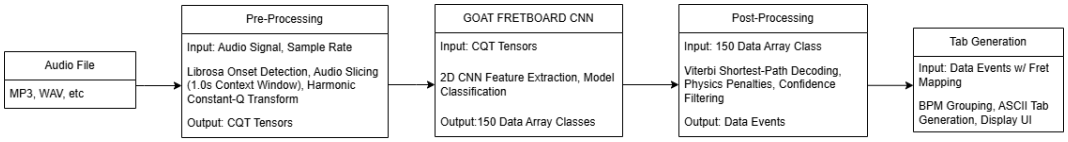

Figure 3: End to End system

To compare the difference in performance between the sequential pipeline to an end-to-end. An end-to-end system has been built from scratch. Rather than using the data from Basic Pitch, this model uses a CNN to take audio input and return guitar (string, fret) output. Using Librosa onset detect on the wav file the model finds spikes in the audio and records the timestamp of the start of the note being played. For every detected note, a 1.0-second audio slice is extracted, centred on the target note, with the surrounding audio providing context for the CNN. This acts as a context window so the model has knowledge of previous notes and next notes so can deciphers what note it is currently playing as. A 1-second slice was chosen to balance sufficient musical context required and to have a lightweight input. This mirrors the same approach taken in fretboard CNN which contained a context window 5, applying the same strategy into the audio domain.

Audio signals are temporal and sequential, unlike images which are based on edges and shapes, this poses a challenge for convolutional neural networks to process. Therefore the audio slices created are converted into Mel Spectrogram. This turns the audio into 2D images, which are preferred by CNNs.

The Mel Spectrogram tensor is then fed into the Goat Fretboard CNN model developed for this system to evaluate the spectrogram and output of 150 classes where it decipher which string and note are being played through the formula 6 strings * 25 fret = 150 classes. The model includes a confidence list for the prediction the note being detected being accurate. The model then selects the highest confidence note in the list

The Mel Spectrogram was replaced with harmonic CQT, which is used in the Basic Pitch model as it can carry out better pitch extraction to detect the fundamental frequencies in audio. The 1.0 second audio slice is turned into a harmonic Constant-Q Transform tensor, this is to create a multi channel visual representation of the frequencies and pitch to help the CNN to eliminate pitch redundancy improving the fret/string mapping.

The model is trained on the GOAT dataset, which contains 5.9 hours of annotated audio which ran till 332 epochs with early stopping, with the model peaking at 150 epoch, plateauing at $\tilde{2}9\%$ where the learning rate fell 8 times down to near zero. This suggests that the model architecture was the bottleneck rather than the training. The model was trained using the GOAT dataset with an 80/20 train/validation split. Training used the Adam optimiser with a learning rate of 1×10^{-3}, and batch size of 64. Full hyperparameters list present in Appendix A, Table 6.

4.5 Viterbi

Both the models use Viterbi [30] to find the best fret/string path to choose. This function converts the probabilities into a log space ensuring the number are large enough and do not fall to zero (underflow). Viterbi solve the core problem of picking the correct path through the 150 possible string/fret positions that is both locally accurate while still being globally coherent. A simpler alternative would be to use greedy decoding which is used in the heuristic algorithm. This can't recover from early mistakes as if it picks a bad position, then all subsequent notes will be based on that bad region. Viterbi considers all 150 output classes at every time step simultaneously and can pick positions that it is slightly less confident if it leads to a better overall output.

The transition penalty between consecutive positions is defined as:

$$\text{penalty}(p_i, p_{i+1}) = \begin{cases} -2.0 & \text{if } s_i \neq s_{i+1} \quad \text{(string change)} \\ -0.5 \times (|f_i - f_{i+1}| - 4) & \text{if } |f_i - f_{i+1}| > 4 \quad \text{(large fret jump)} \\ 0 & \text{otherwise} \end{cases}$$

where $p_i = (s_i, f_i)$ denotes the string and fret at timestep i. The string change penalty reflects the physical difficulty of crossing strings mid-passage, while the fret jump penalty activates over 4 frets as this exceeds the typical span of a guitarist's fretting hand in a single position. These penalties ensure Viterbi favours paths that model realistic human playability rather than selecting positions based solely on classification confidence.

Two matrices are stored, one of them holds the current highest running scores and the other stores the notes that led to that high score. The function runs a forward pass where it compares the 150 current positions to the 20 previous candidates and converts the class into fret and string. For each possible solution it applies a penalty for cases such a large number string jump and based on the physical fret difference. stores the current winning score and stores the index to reach the winning score. Finally it carries out a backward pass, it finds the state with the highest result, and in the end it returns the optimal path.

4.6 Pitch-based Output Masking

The end-to-end system processes audio spectrogram and output probabilities across the 150 classes, but since it uses Librosa onset detection rather than a start of the art pitch detection model, many of its notes are not being detected and the high confidence prediction correspond to physically impossible string/fret combinations. The `mask_by_pitch` solves this by using Basic Pitch to detect the MIDI pitch of each note similarly to the sequential model, then masking out any of the 150 that cannot produce that pitch. This reduces the reliance of the model from what note on what string/fret to which string/fret for this known pitch. A fallback is included if the mask by pitch does not match any of the classes, keeping the original unmasked prediction by the CNN.

4.7 Tab Parsing for Export

The purpose of the tab parser is to take the ASCII tab and convert it into structured data which can be used to export both into PDF and GP5. This is to ensure the modified tabs in the tab editor can be used for exporting.

The tab parser groups the sections of tablature into blocks by filtering out any spaces and looks for bar lines or string letters. The parser scans the 6-line blocks, with added feature of safety checks to prevent the software from crashing when there are errors including a missing dash. The numbers are logged and checks identifies if it is a double digit by also checking the next space. A time offset is also included to the blocks as to identify the order to be stored. This returns a dictionary containing a mapping of notes to the string and fret with the timestamp.

4.8 Exporting

The exporting features to export in either PDF or GuitarPro5 (GP5). The system allows users to export to pdf as it would provide users an immediate option to view the tab generated without requiring additional tools. GuitarPro5 was chosen as an export format because it is widely supported by tab editing software, giving users the flexibility to continue working in their preferred application. MusicXML was not implemented as GuitarPro5 was implemented as it is better for guitar tablature whereas MusicXML is more broad, and similar to MusicXML is supported by most notation software.

5 Design

5.1 System Architecture

The system uses a Model View Controller Architecture. The view was developed using Python's library PySide6 GUI [31] which provides the visual representation to the user of the guitar tablature. The purpose of the controller is to manage the worker thread. The worker QThread supports features such as tab editing, as the display must update to reflect user corrections. The model includes the backend containing the guitar transcription system to process and develop the tabs for the user. The separation of view and controller from the model ensure the system is modular and enables the backend to be tested independently.

5.2 UI Design

To display tabs and provide user-friendly interface for the user to modify and view tabs, PySide6 is used to display the tabs in a desktop application. This ensures the audio is processed locally rather than being run on servers protecting from issues with copyright and privacy. A desktop application also has the benefits of having lower latency than a web application. PySide6 was chosen over other desktop application due to its modern UI compared to alternatives such as Tkinter that has an outdated look.

5.3 Tablature Generation

The requirement for the tab generation currently is to create an ASCII tab based on the information provided by fretboard mapping. A command line interface (CLI) was developed to facilitate testing and to verify the output accuracy. This gives the user the option to run

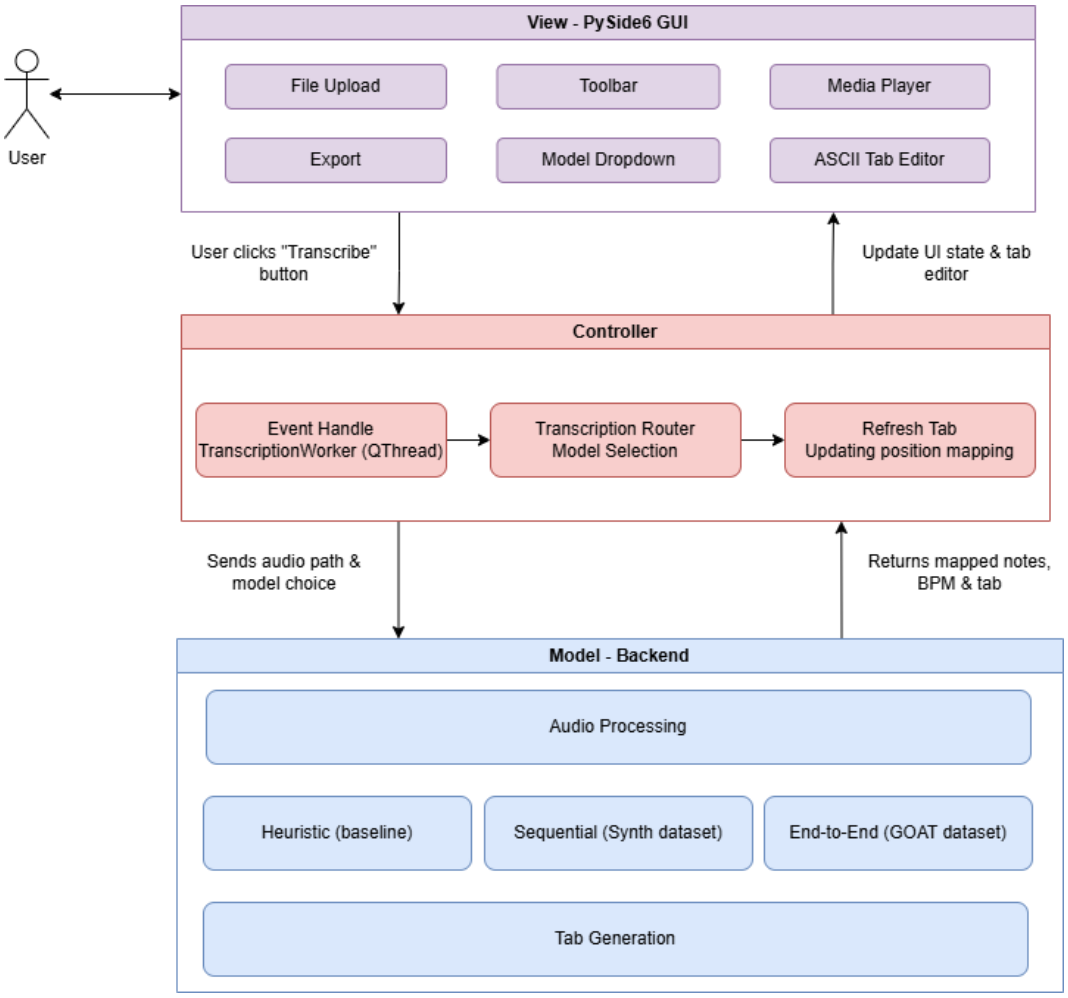

Figure 4: Component Diagram of MVC architecture

the program on their command line interface rather than having to use the graphical user interface if required.

A text editor is used to display the tabs in the GUI. This version allows the user to edit the tabs and provides various features which are unavailable on the CLI version. The option to edit the tabs including to delete the note offers the opportunity for the user to correct the tabs generated by the system

5.4 BPM Detection

Beats Per Minute are essential in grouping the tabs into bars for display. Librosa [32] `beat_track` is applied which estimates the tempo of the onset audio signal. The BPM detected is used to compute the duration 16th-Note subdivision: second_per_slot = (60.0 / BPM * 4) / 16. This allows it to be displayed into an ASCII format using a bar "|" in

the correct intervals. This function tends to perform well for monophonic audio but then produces less reliable results with polyphonic audio when there are multiple notes being played at the same time.

5.5 Confidence Scoring & Traffic Light Design

The end-to-end model developed includes the features to score the confidence of the model's prediction on strings/notes. The confidence data is presented to the user through a 'traffic light system' where the string number is displayed in green, amber, and red:

- Green - confidence score of over 0.6
- Amber - Confidence score of over 0.35
- Red - Confidence score below 0.35

The aim of this feature is to guide the player on notes which the model is unsure about, enabling the user to update the guitar tabs to ensure they are accurate. To achieve "Green" the model needs to score 0.6 and not a higher point such as 0.9+ is because this would lead to the model predict most the notes as "Red" or "Amber", even if the chances of the model predicting are accurate. This would also make the feature useless for the user and would not provide vital information to them. The scores were decided by tuning the model to ensure the results fit the actual probability of the accuracy of the note of the end-to-end CNN.

6 Evaluation

6.0.1 User Workflow and Interface Features

A user would go about using the tool by first following setup process in the README on the project repository. Once the tool is installed a user would select the audio file (mp3, wav, etc) and can choose the model to run. The user would then select the transcribe button, this will take the user to another page. After a few seconds, the lightweight model will reveal the tabs generated with the option to listen to the original audio and compare to the audio of the MIDI generated this connects to the ear training usage where the user can hear the difference and based on this information the user can edit the tabs and make the changes required. The modularity offers the opportunity to swap to better model. Users can experiment the end-to-end model to a heuristic model and compare the outcomes.

The model selection option was provided so a developer and users could select and try the various models with the GOAT fretboard CNN including the option of the 'traffic light'

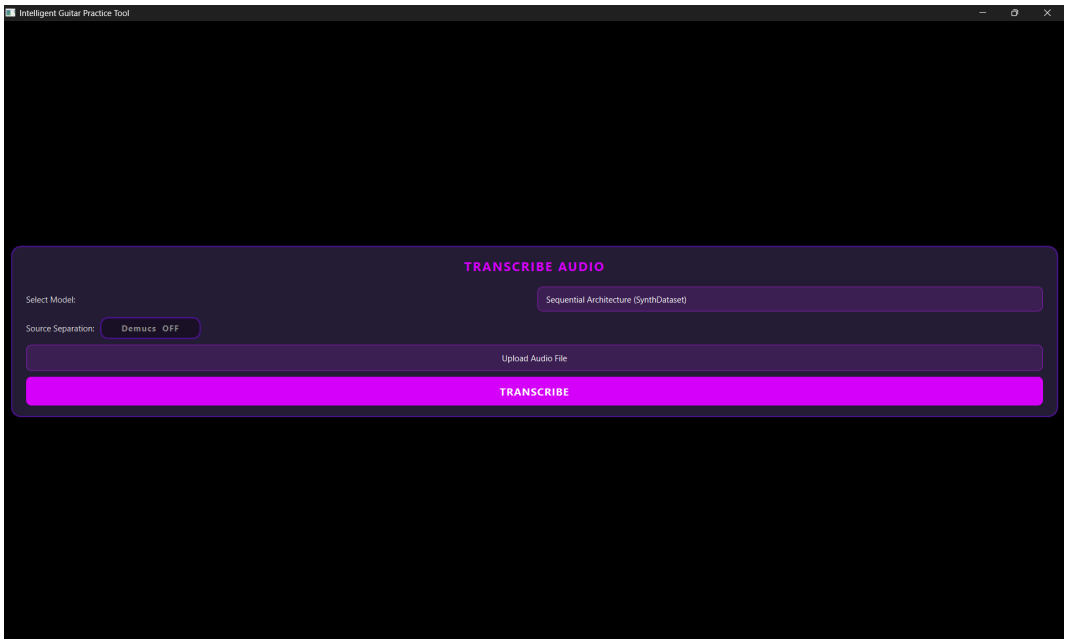

Figure 5: Start page

confidence, this serves for both technical and education purposes. Although outside of that one option there are no major distinguishable differences in the model and would have been better if more features were available to do that.

The editing toolbar has 4 options which the user can carry out: increase fret, decrease fret, add note, and delete note. The increase/decrease fret allows user to update an existing notes fret position. The add note creates a new note set to 0 which can be modified using increase or decrease fret. The delete note will remove a note.

The selection of including a 'traffic light' based confidence over numerical confidence score was chosen as it would be easier for player's to know the confidence at a glance based on the colour display rather than having to interpret the difference between the scores. It also made it easier to visually display the score to the user in the UI.

Another issue noticed when evaluating the models was the challenge of modifying the tabs, which led to alignment crash issues. To encourage users to modify the tabs while not leading to any crashes. The toolbar was developed to user options such to increase/decrease fret, delete note, add note in a safe approach.

To allow users to know what notes need to be changed the user has the option to compare the original audio with the MIDI playback designed specifically for ear training use case.

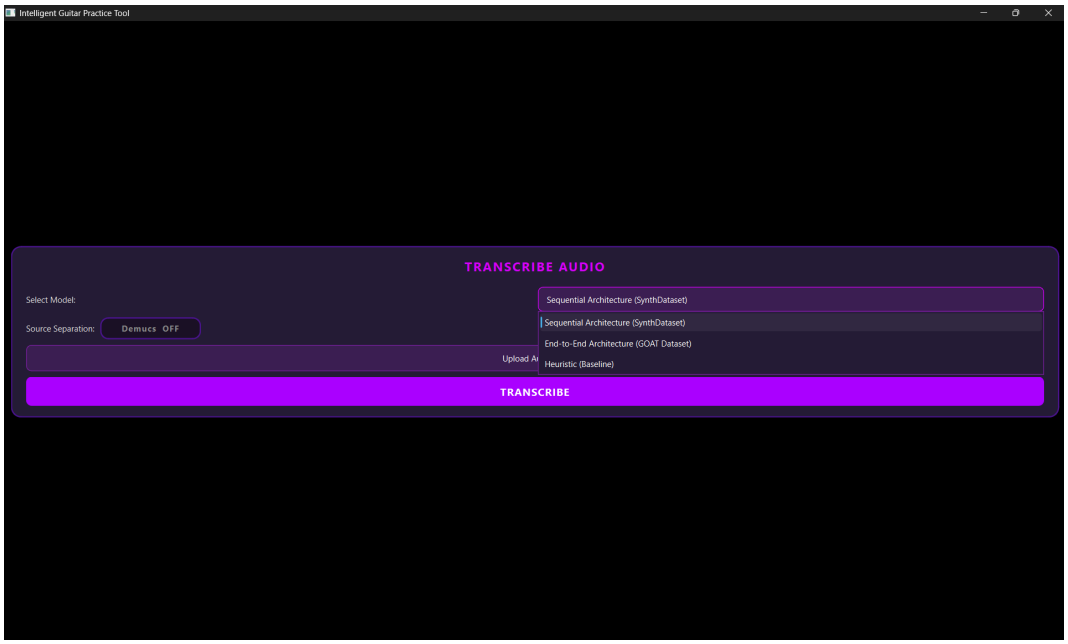

Figure 6: Model selection

6.0.2 Manual Testing and Quality Assurance

To ensure the software does not crash, high level testing was conducted simulating how a user would use the tool and to recognise the issue of the editing tabs breaking and losing alignment. The safety editing toolbar was developed for this reason, to further protect from breaking, the tab parser includes safety checks checking for missing dashes and double digits. Manual testing was conducted on monophonic, polyphonic, and multi-channel audio.

To evaluate the ability of the edit tab tool, the model faced issues when editing the tabs it ran into issue with alignment and this could lead to errors in the software which could crash it. To improve on this a tool bar was developed which allowed users to safely delete, add, increase and decrease an integer string. This enables to system's use case as a practice tool despite the errors with the AI prediction.

The CLI version was developed specifically to test backend output independently from the GUI. This also made it easier to test and view the results of the tablature early on the project. Exported files such as PDF and GP5 file outputs in MuseScore were tested to ensure they render properly in third party software.

Due to time constraints, automated testing, such as unit tests and integration tests were not implemented but would be a priority for future works to ensure regression-free updates to the pipeline components.

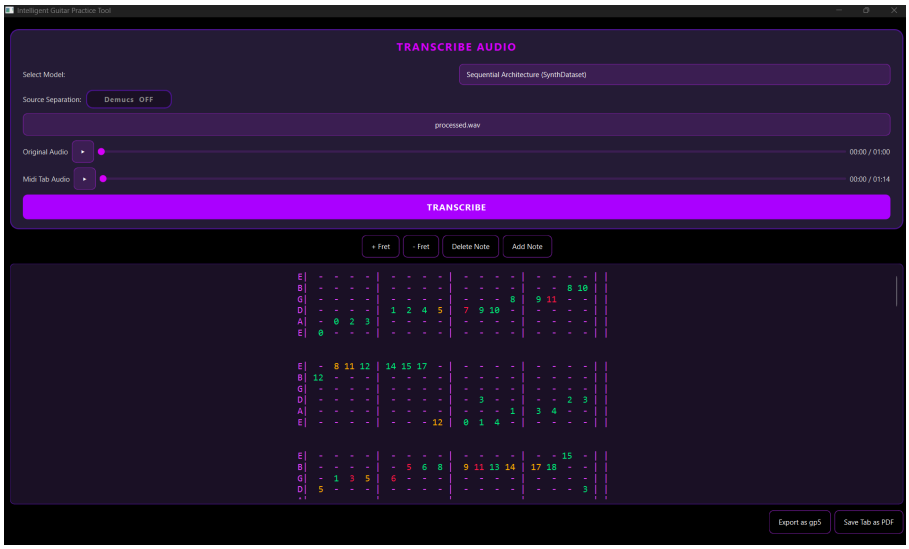

Figure 7: Display of the traffic confidence score in end-to-end system

6.1 Qualitative Analysis

6.1.1 Transcription Behaviour Across Audio Complexity

The three models were evaluated on monophonic, polyphonic and multi-channel based audio to demonstrate both the abilities and drawbacks of the models. The monophonic audio evaluates the models' performance to single-note passage like guitar scales. Polyphonic describes the models performance to multiple notes playing at the same time like guitar chords and fingerpicking patterns. Multi channel is the most challenging situation where the model is being evaluated with multiple audio sources mixed together like in most music. The sequential model results will be displayed for this analysis where it will run the audio files and a snippet of the tab generated will be displayed with the actual tab to show how the model performs to the actual tablature in the different circumstances.

Monophonic - The monophonic audio is the harmonic minor scale. The model performs well on the monophonic audio where the errors are mostly flagged by red, symbolising low confidence. Another issue noticed was that the system would occasionally place two notes in the same time position. This can be seen in the first bar where the 0 (open note) on 6th string and 2nd fret on the 4th string are placed together.

Polyphonic - The polyphonic audio used is Autumn Leaves by Frank Sinatra. The piece is played in fingerstyle, requiring the system to capture multiple notes in the same line. The model performs this at times well for example in the first bar it was able to capture the notes 1st fret 2nd string and open note 5th string. The number of notes coloured red also increases,

showing the model struggles to capture the correct note. Tablature accuracy decrease when the model is presented with multiple simultaneous notes.

Multi-Channel Based audio - The multi-channel based audio used is Knockin' on Heaven's Door by Bob Dylan. The audio consists of vocals, drums, bass, and guitar. To be able to transcribe the audio, the model uses Demucs to isolate guitar from the other sources. The tablature uses open chords specifically G, D, and Am chords. The results show the model struggles to handle open chords with the model missing the open notes that are being played from the audio. The usage of Demucs introduces noise such as amplitude reduction, and spectral bleed which leads to the Basic Pitch note detection being affected leading to the transcription being less accurate.

6.2 Pitch Detection Evaluation

In the appendix, figure 11 displays Loss Curve of Goat Fretboard cross entropy loss. The cross-entropy loss decreases and starts to converge at 200 and becomes flat and the validation accuracy peaks at around $\tilde{1}10$ before declining.

One of the datasets that is being used to test the performance of the audio-to-MIDI pipeline is GuitarSet: a dataset designed for guitar transcription, aiming to address the challenge of the polyphonic factor of audio tracks. GuitarSet consists of 3 hours of content in total, picking up audio through a hexaphonic pickup stores an output of the separate channels for each of the six guitar strings. The current evaluation uses the mono-microphone setup instead. This mono-microphone setup was chosen over the hexaphonic setup to evaluate real-world performance, as most users will upload audio which will not be from a specialised hexaphonic pickups. This will provide a more realistic assessment of the system's applicability. Metrics tracked include the number of notes found that were correct and the notes that did not match the annotation pitch. Calculating metrics including precision, accuracy and recall using mir_eval [33]. The results outline the effect of pre-processing degrading the performance of the pitch detection model, likely because Basic Pitch expects noisy audio rather than a cleaned result. For this reason, the pre-processing will be removed from the pipeline. The pre-processing substantially reduces the number of predicted notes which leads to a lower chance of the note being correct. The high-pass filter may remove low-frequency fundamental information needed for accurate pitch detection by Basic Pitch as it loses the fundamental information needed to make accurate predictions. Normalisation also causes this issue as it adjusts the amplitude of the signal to a constant target, changing the expected input for Basic Pitch. The post-processing improves the performance of the model and removes additional notes when mapping and will be included in the final version.

Configuration	Precision	Recall	F1-Score	Predicted Notes
Baseline (Basic Pitch)	0.806	0.853	0.821	62659
Pre-processing	0.794	0.687	0.716	46256
Post-processing	0.877	0.812	0.833	52860
Full Pipeline	0.859	0.642	0.711	38396

Table 2: Performance comparison of pipeline configuration. Note: The Ground Truth of GuitarSet contains 62,476 actual notes.

In the pipeline configuration the preprocess predicted notes performed worse missing 16,403 notes compared to the baseline model. The preprocess performs worse as the Basic Pitch Model is trained on noisy audio data and cleaning it might impact the model training.

The fretboard mapping evaluation shows the CNNs outperform the heuristic model by approximately 10 percentage points. This improvement is attributable to the CNN's ability to learn contextual patterns of the data to be able to map the correct string and fret based on the context window making the model learn the patterns. The heuristic model relies on the greedy approach to select the nearest correct note, this fails to acknowledge the complexity of audio and might only perform well on monophonic audio.

The baseline Basic Pitch score published score 0.84 is comparable to the score achieved during the test where it achieved a F1 score of 0.82. This score was reached after fine tuning the Basic Pitch model to reach the score.

6.3 Fretboard Evaluation

To evaluate the accuracy of the models, the GuitarSet dataset was utilised as the ground truth. One of the challenges when comparing the actual notes (dataset result) and the predicted notes (model results) is the challenge of aligning to the onset time. To address this, a time threshold was used, and the time matched the closest to the onset time of the actual note, avoiding issues of incorrect matching by implementing greedy matching algorithm. The note is classified as a True Positive (Exact Match) only if the pitch, onset time, string index, and fret index all match the ground truth.

Metrics of Average String Error and Average Fret Error were calculated to quantify the distance between the predicted and actual position.

To ensure a fair comparison and prevent data leakage, all models were evaluated on the same test split of the GuitarSet dataset. As shown in Table 3, the fretboard CNN significantly outperformed the baseline. These results highlight the data efficiency of the proposed sequential

pipeline. While the end-to-end TabCNN struggles to generalise from the limited size of the training set, the model specifically fails to detect note events, predicting only 2898 notes out of the 8715 ground-truth notes. In contrast, the modular approach leveraged the capabilities of Basic Pitch, allowing the Fretboard CNN to focus exclusively on the positional mapping.

In the appendix, figure 8 displays the confusion matrix for GOAT fretboard CNN and figure 9 likewise for Synth Fretboard CNN. The purpose of the graph is to see if the model predicting the adjacent string. The Synth CNN showed a consistent bias toward predicting the adjacent lower string across all six strings, with the most pronounced case being the 6th string, where the model predicted the 5th string 40% of the time. This bias may stem from SynthTab's synthesised audio, which lacks the timbral variation between strings present in real recordings. The Synth Fretboard CNN was predicting the lower string for example rather than string 2 it would predict string 3. The GOAT fretboard CNN showed a mixed representation where it would predict either to the string to its left or right by one, leading to the confusion matrix to be more spread out. Neither model could be said to be performing better than the other. The results could outline the performance of the model lowered due to this rather than pitch redundancy. The GOAT dataset larger error could also be due to the domain difference where in GOAT is trained on recordings of electric guitar whereas the GuitarSet evaluates on acoustic guitar. The GOAT CNN scores 85.5% accuracy on high-E string to 54.2% accuracy of the low-E string, this shows the model performs better on high strings, this can be due to the timbres of the notes being more distinct.

Model	**Accuracy** (%)	**Avg. String Error**	**Avg. Fret Error**
Simple Heuristic	49.6	0.81	3.88
Fretboard CNN	60.3	0.43	2.01
TabCNN	60.3	0.29	1.52
End-to-End (GOAT CNN)	17.4	0.31	1.45

Table 3: Fretboard Mapping Evaluation. Accuracy denotes the percentage of notes mapped to the exact correct string and fret.

7 Discussion & Interpretation

7.1 Comparison of Model Architectures

Both the sequential (Synth dataset) and end-to-end (GOAT) models use different datasets from the evaluation, although this was one of the reasons for the results to be lower, it also

Model	Dataset	Metric	Conditions	Result
Simple Heuristic	GuitarSet	Direct Match	Note+String+Fret+Onset	49.6%
Fretboard CNN	GuitarSet	Direct Match	Same as above	60.3%
End-to-End	GuitarSet	Direct Match	Same as above	17.4%
Hybrid End-to-End	GuitarSet	Direct Match	Same as above	60.2%
Fretting Transformer	clean MIDI	Tab Accuracy	Clean MIDI Input	∼72%
TabCNN	GuitarSet	TDR	Correct Pitch Assumed	89%

Table 4: Comparison with Benchmarked models

shows how the model generalise to completely new data, similar to what a user might upload to the transcription tool. This also produces results that are not inflated by overfitting.

The end-to-end model performs significantly worse than the benchmarked results from the research and compared to the CNNs developed, likely due to data size being used to train the model being smaller where it is trained on a smaller size of files from files from 1-4 (300 files) and leaving the files ending with 5 (60 files) to be used for testing, The TabCNN research reported an accuracy of 89% and this differ vastly by the one conducted here as the TabCNN research was related to disambiguation rate assuming the correct pitch has been detected, while the test here aims to takes into account both the pitch detection and fretboard mapping are accurate. The test conducted here is much stricter where model is being evaluated on its ability to detect the notes and predicting the string/fret played. The fretting transformer and TART operate under different conditions which make the results incomparable.

The end-to-end GOAT CNN failed to detect the majority of notes where it detected quarter of the notes from the GuitarSet. This leads it to achieve a score of 67.9% in detecting the fret/strings from the 25.6% notes that were detected. The pitch detection failure is likely due to the use of Librosa onset detection, which is primarily a heuristic algorithm approach to detect notes by finding the peaks in audio. To further improve the model one of the methods made is to increase the audio slice from 0.2 to 1 seconds, this provide the model with more context rather than just predicting from the current note, it can use the musical pattern before and after the note where we are predicting the string/fret of the middle note in the audio.

To improve the performance of the end-to-end system performance at pitch detection, pitch masking was added to the system this essentially turned the end-to-end model to a constrained hybrid model. The Hybrid end-to-end system was developed and evaluated on the GuitarSet test set. The system differs from previous end-to-end model by using Basic Pitch for onset and pitch detection, feeding a 1-second Harmonic CQT window to the GOATFretboardCNN, applying `mask_by_pitch`, and use viterbi decoding. Leading the results to closely match the sequential model with a score of 60.2%. This makes the system closer to the sequential

model than to an end-to-end model. This validates the sequential pipeline's core design of separating the pitch detection from the fretboard mapping is not just convenient but is necessary given the current data sizes and model capabilities.

7.2 Limitations

7.2.1 Model Limitations

The models are evaluated using the GuitarSet dataset which has bias as it contains recordings specifically of acoustic guitar. The test does not evaluate the performance of the models on other guitar types. The dataset cannot evaluate whether the models handle different timbres, such as those produced through distortion. This means one cannot conclude if the model would perform well in the real world due to modern music relying heavily on effects and processed guitar, so the evaluation can only apply to a niche application of clean acoustic audio rather than a general AMT. One approach is to evaluate the model on a more diverse dataset which includes challenges of detecting notes from high distortion or low pass guitar. This could be evaluated through cross domain with the Synth dataset. Another approach is to apply VST tools to the annotated MIDI to data augment the GuitarSet. To show fairer result of the model's performance it should be evaluated on an electric guitar dataset rather than GuitarSet but this choice was made as GuitarSet is the standard option of comparison and would display realistic challenge of the different guitar types.

7.2.2 Evaluation Limitations

For an instance to be classified as true positive, the onset, pitch, string, and fret must all match. This underestimates the practical usability since a note that was one fret off is still mostly useful for a guitarist when practising and making changes. The model does not include a test set on the fretboard CNN for SynthTab dataset and the GOAT dataset therefore the 87% and 28% validation accuracy cannot be directly compared.

7.2.3 Tool Limitations

The system showcased how an AMT system could successfully be used as a Guitar Practice Tool, but due to some limitations in the models it could not replace current tools. A primary issue identified was the lack of a mechanism for users to verfiy whether their corrections are accurate, with the accuracy of the prediction still averaging around 60.3%, it cannot be used as a standalone tool for tablature generation but should rather be used as a tool to verify if a

tab generated seems accurate. The end-to-end GOAT CNN main issue was due to the usage of the Librosa onset detection which was only able to detect 25.6% of the GuitarSet notes.

Another challenge noticed as an ear training tool is the issue of MIDI audio generated not being synchronised to the original audio where the MIDI audio player is much faster compared to the original audio. This was because the MIDI player did not account for the tempo and note duration leading it to be much harder to compare the exact positions between the audio. To solve this limitation the MIDI player will also use the onset and offset to preserve the original timing generating a more realistic MIDI to the original. Another solution is to include a playback speed so users can slow down the audio and can identify notes more easily.

Due to the model generating transcription it cannot tell if the changes made to the tab are correct or not due to the system not containing the actual tabs to compare against. This makes it more challenging for users to know if the modifications are making it harder to be used as an ear training tool.

7.3 Practical Utility & Design Trade-offs

Given the accuracy of 60% fretboard accuracy the user can compare their own attempts and correct using the toolbar which makes the tool still useful as a learning approach, especially with the addition of the confidence traffic light guiding their attention. Introducing challenges during practice leads to deeper learning and better long-term retention than passive study. The system creates this experience by making the users actively identify and correct errors rather than passively reading a perfect tab.

While most models are optimised for transcription accuracy, this dissertation prioritises practical usability as a practice tool as the other models are evaluated using TDR and F1-Score, but for a practice tool the relevant challenge is whether the model is accurate enough for a learner to work with. A model with 60% accuracy that lets users make changes might be more valuable to a model with 90% for the education purposes [34]. This concept is supported by the theory of desirable difficulties, where introducing challenges during learning leads to stronger knowledge long-term than passive study. The system follows this behaviour by forcing users to actively engage with the audio, identify errors through the confidence traffic light, and correct the tabs.

All models run locally on the users device, this protects music copyright as the music files are not being processed in a server side system compared to online commercial tools like Songsterr which sacrifices privacy and requires internet access. The commercial tools likely achieve higher accuracy through proprietary datasets and engineering resources. The model developed in this project are lightweight so can create tabs in few seconds saving users time

waiting for the tab to be generated. The design of the sequential pipeline model enables functions to be replaced such as Basic Pitch being replaced by an MT3 rather than having to rewrite the fretboard mapper.

Although the model scores 60% accuracy it is still useful due to the ability as tool including editing and confidence feature.

7.4 AMT in the Wider Context

Currently the trend in AMT is moving towards larger models (MT3, transformers) [35], relying on GPU processing and cloud-based services. This creates an accessibility barrier as not everyone can run these models or can justify cloud computing. This dissertation supports an alternative approach of a lightweight, interactive system. The tool allows users to upload audio, this cannot be done in tools like Yousician. This also opens up as a tool available for self-taught guitarists, who don't have access to music tutors or learners who can't afford Yousician subscriptions or professional transcribers. It also enables transcription of music not available in curated libraries.

Commercial Products such as Songsterr and Moises are guitar transcription tools that are popular but are closed-source and cloud dependent. The system developed here is open and local, therefore allowing researchers to audit, swap components or build on it. Researchers can replace Basic Pitch with an improved pitch detection model without rewriting the fretboard mapper. The modular MVC architecture means the GUI, backend models and export pipelines are independently testable and replaceable.

8 Project Management

8.1 Project Plan

An agile process was proposed for the management of the project, similar to any other software development. Adopting an agile methodology through weekly supervisory meetings, which ensured alignment with the project's core aims.

In the first semester, focused heavily on the development of the model, creating a transcription for the system. The project had weekly supervisor meeting to discuss and applying the feedback into the model. The project ran smoothly this semester where it was following the expected timeline of the transcription.

In the second semester, it was assumed there would be more time to work on the project due to it being only a 50 credit semester, but other circumstances reduced the time available. Sections that were removed included user evaluation which would have been vital in providing a better understanding on the UI/UX making changes required to fit the user needs, but would have cost a substantial investment of time to prepare and would have required guitarist, to accommodate for this change, there was a significant focus into performance evaluation and qualitative evaluation which looked into both the models performance and the user experience perspective. In retrospect the full formal study could have been replaced by an informal session which would have targeted on UI rather than performance.

Due to the end-to-end system development (Task J) overrunning its scheduled time, technique classification was removed from in order to put more attention into transcription accuracy than expected as it was unexpected how much the fretboard mapping would dominate development time.

The GOAT CNN plateauing at 29% shows the architecture developed was not the only issue. This shows the training set was insufficient, which was why a test set was not created so as not to further decrease the size of the training set. If started again, the model would have been allocated more files for training through data augmentation. One of the factors could be due to the output class being 150 but only containing $\tilde{3}00$ training files. Not every string/fret combination appears in the real music. Predominantly the first 5 frets dominate guitar playing, meaning many of the 150 classes may have very few or zero training examples. If the example is never seen then the model is less likely or never outputted.

Future projects would include more contingency time specifically to handle model iterations, hyperparameters tunings and setting up HPC ada and handle HPC queue times which were underestimated, and would prioritise a section for data augmentation in the pipeline.

The project is also being tracked using the DevOps tool, Atlassian JIRA. The project management tool has a stronger focus on individual tasks compared to the broad overview of the Gantt chart. Large scale requirements have been set as Epics and problems have been broken down into tasks, improving the modularity of the development and creating a measurable progress tracker.

The appendix figure represent the final project gantt chart which introduces new tasks that needed to be carried includes the safe toolbar editor, traffic confidence system, and audio and MIDI player features for improving the training session. There also includes a 3 week block from December week 3 to January week 1 which was not planned for, during development. In the future, buffer time will be considered in-depth for exam season and holidays.

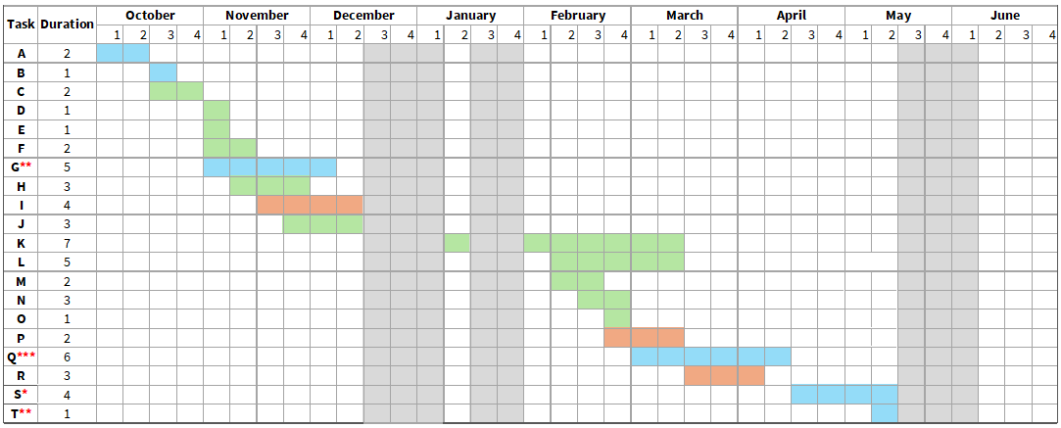

Figure 8: Gantt Chart Final Outcome

ID	Task Description	Start	End	Duration
A	Complete Project Proposal	Oct W1	Oct W2	2 weeks
B	Complete Ethics and Data Management Plan	Oct W3	Oct W3	1 week
C	Foundation & Prototyping (Audio Loading, Basic Pitch)	Oct W3	Oct W4	2 weeks
D	Basic Tab Generation (Single Notes & Simple Melodies)	Oct W4	Oct W4	1 week
E	Multi Track Audio Separation	Nov W1	Nov W1	1 week
F	Improve Tab Generation (Chords, Polyphonic Audio)	Nov W1	Nov W2	2 weeks
G	Write & Submit Interim Report (10% of grade)	Nov W1	Dec W1	5 weeks
H	Implement Sequential Model (CNN Mapping)	Nov W2	Nov W4	3 weeks
I	Model Evaluation	Nov W3	Dec W2	4 weeks
J	Develop End-to-End Model	Nov W4	Dec W2	3 weeks
K	User Interface Development	Jan W2	Mar W2	7 weeks
L	Safe Editing Toolbar	Feb W2	Mar W2	5 weeks
M	Export Functionality for File Format	Feb W2	Feb W3	2 weeks
N	Traffic Confidence Score System	Feb W3	Mar W1	3 weeks
O	Audio Player and MIDI Player	Mar W1	Mar W1	1 week
P	Qualitative Evaluation	Mar W2	Mar W3	2 weeks
Q	Write Final Dissertation (75% of grade)	Mar W1	Apr W2	6 weeks
R	Final Code Polish & Demo Video Creation	Mar W3	Apr W1	3 weeks
S	Prepare Presentation & Demo for Assessment	Apr W3	May W2	4 weeks
T	Final Presentation/Demonstration (15% of grade)	May W2	May W2	1 week

Figure 9: Complete Project Plan

Type	Colour
On-hold	
Documentation	
Development	
Testing	
Major deadlines	*

Figure 10: Task highlight key

8.2 Meetings Records

During the initial stages of the project, weekly supervisory meetings were conducted, where ideas were exchanged on how to improve the AMT system. Meeting records were maintained to ensure supervisor feedback was implemented in the code. This includes scope refinement and aims to ensure feasibility within the time constraints. A key outcome was prioritisation of transcribing standard tuning. This decision is required due to the limited dataset related to different tunings, allowing the model to be evaluated without introducing additional complexity with the various instrument configurations. Following an agile methodology, through the weekly supervisor meetings allowed us to catch issues early on such as bug fixes and features. For example the in-line editing was switched to the toolbar-based editing after catching the risk it entailed.

Meeting Record: Progress & Feedback

Agenda: Data Management Plan, Ethics Form, Research Review.

Progress Update:

- Started work with *Basic Pitch*.
- **Issue:** Fretboard fails to map correctly.

Key Supervisor Notes:

- **Audio Processing:** Focus on "peaks and valleys" in raw audio. use smoothing/grouping. Test with guitar scales.
- **Tools to Check:** Google DDSP [36], PureData [37].
- **Future:** Handle tracks with FX; decide between automated notation vs. user-editable tabs.

References: Serra (DSP Course) [38].

9 Conclusions and Reflections

9.1 Contributions

This dissertation contributes a comparative analysis of sequential pipelines and end-to-end models for guitar transcription. While demonstrating how an imperfect transcription system can be repurposed for the interactive practice for training and learning. The confidence traffic light aims to support learners judgement to carry out active recall and error-driven learning. The feature of MIDI playback comparison supports the paradigm of learn, predict, compare, correct workflow, ensuring users listen to the original audio and compare to the MIDI audio to notice the difference and make the required changes.

To the author's knowledge, no prior work has combined automatic guitar transcription with an interactive correction workflow specifically targeting ear training. The systems reviewed in the related work (TabCNN, Fretting Transformer, TART) all generate tabs with no features for user learning.

9.2 Reviewing Requirements

Based on the requirements set at the start of the project, this section will review them and discuss what has been met and what has not been completed. Development of the sequential pipeline and the end-to-end model was essential for demonstrating how the system would operate. The GUI and exporting features play a crucial factor in shifting from a research experiment to a tool for users to practice ear training and editing the tab. The project has met or partially met most requirements to build the end-to-end system although falls short in achieving the 80% accuracy, being able to achieve F1-Score 0.81 in pitch detection through Basic Pitch but the complete system achieving 60%. During requirements planning, the F1-score target was set at 0.80, representing the estimated threshold for usability as a transcription tool. This target did not account for the specific challenges of guitar transcription, such as pitch redundancy. 60% fretboard accuracy still serves the practice tool use case because the user is expected to correct errors, not receive perfect tabs. Other features have been developed such as traffic confidence system and MIDI player to specifically improve the corrective workflow.

9.3 Legal, Social, Ethical and Professional Issues

This project follows strict legal and ethical responsibilities that need to be taken in order to follow the right standards. For instance, it was important to ensure the dataset that was used had not been made publicly available in the repository, so as to follow the rules regarding copyright of the dataset that was being used. Another related case is when access to the GOAT dataset was requested, due to the dataset containing covers of popular songs. Another case that one needs to be aware of is if users upload copyrighted songs to the tool, this would generally not be an issue, as currently the system does not download the song, but it might need to be taken into consideration if reinforcement learning is implemented to improve the model's performance by using the audio the user provided. Similar methods of reinforcement learning have been applied by models such as MusicRL [39] to align music generation with human preference, while high-dimensional text-to-audio models like MusicLM [40] highlight the growing capabilities and copyright complexities of generative AI. An ethical challenge that the AMT tool faces is the tab generated might not be accurate relative to the original recording which could lead to users' learning a song incorrectly, to mitigate this risk, the system will explicitly include disclaimers regarding potential transcription inaccuracies.

No.	Requirement	Met or Not	Result
1	Research and benchmark existing models (Fretting Transformer, TabCNN)	Met	TabCNN was benchmarked and compared against developed models.
2	Develop sequential pipeline models (heuristic baseline and fretboard CNN)	Met	Simple heuristic and fretboard CNN model developed using sequential pipeline approach.
3	Develop end-to-end model using 2D CNN on GOAT dataset	Met	End-to-end model using GOAT dataset with 2D CNN architecture completed.
4	Develop software UI with tab generation, PDF export, music software export, and tab slicing	Partially Met	UI developed with tab generation and export features (PDF, GP5). Tab slicing not implemented due to time constraints.
5	Generate guitar technique classification (bends, pull-offs, etc) from audio feature. Training the model using SynthTab's [10] dataset and GOAT dataset [11] evaluating novel features of the tool.	Partially Met	Focus remained on improving baseline performance. Qualitative evaluation conducted on monophonic, polyphonic, and multi-instrument audio.
6	Evaluate models and compare results (F1 > 0.8, technique detection)	Partially Met	Model evaluation completed with TabCNN comparison. F1 score <0.8 due to pitch detection failures. Technique detection not tested.

Table 5: Requirements Evaluation Table

The current version faces a challenge with accuracy, which may lead to some users receiving incorrect tabs. One way to address this problem will be to allow users to amend the tabs that are being generated so they can fix any errors that were made, working well as a practice tool. To ensure users can identify cases where there are errors in the tabs, add the option to view the confidence for users to detect uncertainty in outputs. Another case that must be considered is the case of bias in the model from being trained on a specific genre or style of play (fingerstyle, strumming, and tapping). The model should be trained on a diverse training set to reduce bias. As an automated tool, it is not designed to replace professional transcribers but rather to support individual practice sessions, acknowledging that it cannot provide perfect transcriptions across all musical contexts. For the project, one of the concerns with machine learning models include energy consumption and the carbon footprint [41], to manage this situation priority has been placed on using lightweight models such as Basic

Pitch and also building models which are computationally cheaper than other option like a 1D temporal CNN rather than using transformers like fretting transformer [7]. To make the tool physically accessible for users, the heuristic model was designed using Manhattan distance to minimise users' hand movement, thereby preventing users from injuring their hands when practising.

9.4 Future Works

One approach to improve the model would have been to include data augmentation to the dataset. This can be done through the use of PedalBoard by Spotify's research team [42]. Pedalboard creates studio quality effects to the audio, This can create audio samples for rock/metal through distortion or pop audio style. This ensures the model is not just learning from clean audio data and is trained from a variety of styles which can lead it to perform better on those type of genres.

To complete Requirement 5 of having a guitar technique detection process it can be implemented through the use of TENT which would have analysed the pitch contours between the following notes and classify the detection which can be stored alongside each note in the 'mapped_notes'.

Another approach to gain an overall improved results would be to further research into transformers and develop an AMT using transformers, recent discoveries such as MT3 discussed in this topic, leverages transformers to convert MIDI into tablature, using the T5 architecture although this leads to issues of requiring GPU and would not be lightweight option. The factor of the context window still being too small plays a part in convolutional neural networks failing to predict the string/fret to play, while a transformer has the self attention mechanism which can consider the importance of the audio as a whole to the note that is currently being played. This can lead to better fretboard mapping performance. The dissertation focused on using convolutional neural network for because they are computationally inexpensive whereas a transformer would require being hosted on a GPU to perform. Another challenge of the transformer model includes requiring a larger dataset as it can lead to the case of overfitting otherwise, to handle this the dataset would need to be augmented through methods of the Spotify's research tool Pedalboard which was discussed.

To further evaluate and ensure to provide a useful complete project to the end user, The evaluation would have included user evaluation to test the behaviour, UI/UX and the usage of the tool. The user testing the tool would have shown how the tool is used enabling the development of system more fit their needs. The UI/UX could also be updated to the user preference and would improve their experience. User evaluation was not carried out due again requiring significant amount of time to set up and partake in it and it would be difficult

to update the project based on the feedback provided in the tight deadline. The user evaluation evaluation would involve 5 guitarists across different skill ranges, attempting to use the tool to measure tabs usefulness and accuracy to the original tabs from a user's perspective, it would also involve the user evaluating as an ear training tool using the features such as the confidence traffic and MIDI player to make changes from the editing toolbar, based on the feedback modification will be made on the system such as implementing additional features or fine tuning the models.

Finally, this project demonstrates that a lightweight, local run AMT model can be practically designed for a high level of accuracy using a sequential pipeline. By separating the pitch detection from fretboard mapping, the system achieves resilience against specific components failing and allow for future improvements. Such as improving the CNN or training with a larger dataset. While the current fretboard accuracy cannot replace professional transcription, it is sufficient for its purposes as a practice tool with the interactive features.

9.5 Acknowledgements

I am grateful for access to the University of Nottingham's Ada HPC service for the computational resource to train the models. I am also grateful for the access to datasets that were used during this project, including GuitarSet, SynthTab and GOAT.

9.6 Declaration of AI Usage

AI tools were used in the following capacities during this project. Gemini 3 was used to handle compatibility issues when running third-party models, such as TabCNN, and to resolve bugs when training these models on the Ada HPC clusters and evaluating them. Gemini 3 was also used to understand and implement the Librosa harmonic Constant-Q Transform for audio feature extraction in the end-to-end system. Grammarly was used for proofreading and spelling checks to the dissertation. No AI-generated content has been presented as my own original work. All model architectures, evaluation methodologies, and analyses are my own.

Bibliography

References

[1] M. McVicar, A. Pritchard, Y. Sutter, and P. McVicar, "Guitar tab mining, analysis and ranking," in *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, 2011, pp. 341–346.

[2] G. Waddell and A. Williamon, "Technology use and attitudes in music learning," *Frontiers in ICT*, vol. 6, p. 11, 2019.

[3] E. Benetos, S. Dixon, D. Giannoulis, H. Kirchhoff, and A. Klapuri, "Automatic music transcription: Challenges and future directions," *Journal of Intelligent Information Systems*, vol. 41, no. 3, pp. 407–434, 2013.

[4] R. M. Bittner, J. J. Bosch, D. Rubinstein, G. Meseguer-Brocal, and S. Ewert, "A lightweight instrument-agnostic model for polyphonic note transcription and multipitch estimation," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 781–785.

[5] C. Kehling, J. Abeßer, C. Dittmar, and G. Schuller, "Automatic tablature transcription of electric guitar recordings by estimation of score- and instrument-related parameters," in *Proceedings of the 17th International Conference on Digital Audio Effects (DAFx-14)*, 2014.

[6] M. Kaliakatsos-Papakostas, G. Bastas, D. Makris, D. Herremans, V. Katsouros, and P. Maragos, "A machine learning approach for midi to guitar tablature conversion," in *Proceedings of the 19th Sound and Music Computing Conference (SMC)*, 2022, pp. 192–199.

[7] A. Hamberger, S. Murgul, J. Schmidt, and M. Heizmann, "Fretting-transformer: Encoder-decoder model for midi to tablature transcription," in *Proceedings of the 50th International Computer Music Conference (ICMC)*, 2025.

[8] A. Wiggins and Y. Kim, "Guitar tablature estimation with a convolutional neural network," in *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, 2019, pp. 284–291.

[9] M. Good, "Musicxml: An internet-friendly format for sheet music," in *XML Conference and Exposition*, 2001.

[10] Y. Zang, Y. Zhong, F. Cwitkowitz, and Z. Duan, "Synthtab: Leveraging synthesized data for guitar tablature transcription," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 1286–1290.

[11] J. Loth, P. Sarmento, S. Sarkar, Z. Guo, M. Barthet, and M. Sandler, "Goat: A large dataset of paired guitar audio recordings and tablatures," in *Proceedings of the 26th International Society for Music Information Retrieval Conference (ISMIR)*, 2025.

[12] D. Strait and N. Kraus, "Playing music for a smarter ear: cognitive, perceptual and neurobiological evidence," *Music perception*, vol. 29, no. 2, pp. 133–146, 2011.

[13] K. Ng and P. Nesi, "i-maestro: Technology-enhanced learning and teaching for music," in *Proceedings of the International Conference on New Interfaces for Musical Expression*, 2008, pp. 225–228.

[14] M. V. Araújo, "Measuring self-regulated practice behaviours in highly skilled musicians," *Psychology of Music*, vol. 44, no. 2, pp. 278–292, 2016.

[15] M. Mauch and S. Dixon, "pyin: A fundamental frequency estimator using probabilistic threshold distributions," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 659–663.

[16] A. de Cheveigné and H. Kawahara, "YIN, a fundamental frequency estimator for speech and music," *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002.

[17] J. W. Kim, J. Salamon, P. Li, and J. P. Bello, "Crepe: A convolutional representation for pitch estimation," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 161–165.

[18] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[19] J. Gardner, C. Hawthorne, I. Simon, E. Manilow, and J. Engel, "Mt3: Multi-task multitrack music transcription," in *International Conference on Learning Representations (ICLR)*, 2022.

[20] X. Riley, D. Edwards, and S. Dixon, "High-resolution guitar transcription via domain adaptation," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024.

[21] T.-W. Su, Y.-P. Chen, L. Su, and Y.-H. Yang, "TENT: Technique-embedded note tracking for real-world guitar solo recordings," *Transactions of the International Society for Music Information Retrieval*, vol. 2, no. 1, pp. 15–28, 2019. [Online]. Available: https://doi.org/10.5334/tismir.23

[22] Q. Xi, R. M. Bittner, J. Pauwels, X. Ye, and J. P. Bello, "Guitarset: A dataset for guitar transcription," in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 453–460.

[23] J. Paulus, M. Müller, and A. Klapuri, "State of the art in music segmentation and structure analysis," *EURASIP Journal on Advances in Signal Processing*, vol. 2010, pp. 1–13, 2010.

[24] O. Nieto and J. P. Bello, "Systematic exploration of computational music structure research," in *Proceedings of the 17th International Society for Music Information Retrieval Conference (ISMIR)*, 2016, pp. 547–553.

[25] K. Ullrich, J. Schlüter, and T. Grill, "Boundary detection in music structure analysis using convolutional neural networks," in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, 2014, pp. 417–422.

[26] J. B. Smith, J. A. Burgoyne, I. Fujinaga, D. De Roure, and J. S. Downie, "Design and creation of a large-scale database of structural annotations," in *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, 2011, pp. 555–560.

[27] A. Défossez, N. Usunier, L. Bottou, and F. Bach, "Hybrid spectrogram and waveform source separation," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2021, architecture used for Demucs.

[28] H. Heijink and R. G. Meulenbroek, "On the complexity of classical guitar playing: functional adaptations to task constraints," *Journal of motor behavior*, vol. 34, no. 4, pp. 339–351, 2002.

[29] A. Défossez, N. Usunier, L. Bottou, and F. Bach, "Demucs: Deep extractor for music sources with extra unlabeled data remixed," *arXiv preprint arXiv:1909.01174*, 2019.

[30] A. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[31] The Qt Company, "Pyside6," 2024. [Online]. Available: https://doc.qt.io/qtforpython/

[32] B. McFee, "librosa/librosa: 0.11.0," 2025. [Online]. Available: https://doi.org/10.5281/zenodo.15006942

[33] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis, "mir_eval: A transparent implementation of common mir metrics," in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR)*, 2014.

[34] R. A. Bjork and E. L. Bjork, "Desirable difficulties in theory and practice." *Journal of Applied research in Memory and Cognition*, vol. 9, no. 4, p. 475, 2020.

[35] F. Jamshidi, G. Pike, A. Das, and R. Chapman, "Machine learning techniques in automatic music transcription: A systematic survey," *arXiv preprint arXiv:2406.15249*, 2024.

[36] J. Engel, L. Hantrakul, C. Gu, and A. Roberts, "Ddsp: Differentiable digital signal processing," in *International Conference on Learning Representations (ICLR)*, 2020.

[37] M. Puckette, "Pure data: another integrated computer music environment," in *Proceedings of the Second International Computer Music Conference*. Citeseer, 1996, pp. 37–41.

[38] X. Serra and J. O. Smith III, "Audio signal processing for music applications," Coursera, 2019, accessed: 2025-12-06. [Online]. Available: https://www.coursera.org/learn/audio-signal-processing

[39] G. Manco, E. Benetos, and S. Dixon, "Musicrl: Aligning music generation to human preferences," *arXiv preprint arXiv:2402.04229*, 2024.

[40] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi *et al.*, "Musiclm: Generating music from text," *arXiv preprint arXiv:2301.11325*, 2023.

[41] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," in *Proceedings of the 57th annual meeting of the association for computational linguistics*, 2019, pp. 3645–3650.

[42] P. Sobot, "Pedalboard," Zenodo, 2023, accessed: 2026-03-01. [Online]. Available: https://doi.org/10.5281/zenodo.7817839

A Model Data

Parameter	Fretboard CNN	GOAT CNN
Data		
Dataset	SynthTab	GOAT
Input Representation	MIDI Embeddings	Harmonic CQT
Input Shape	Sequence of 11 notes	1.0s audio slice
Train / Val / Test Split	80 / 20 / 0	80 / 20 / 0
Architecture		
Type	1D CNN	2D CNN
Conv Blocks	3	3
Channel Depths	32, 64, 128, 256	32, 64, 128
Kernel Size	3	3
Dropout Rate	0.25	0.5
Activation	ReLU	ReLU
Output Classes	150 (6 strings $\times$ 25 frets)	150 (6 strings $\times$ 25 frets)
Training		
Optimiser	Adam	Adam
Initial Learning Rate	0.001	0.001
LR Scheduler	ReduceLROnPlateau	ReduceLROnPlateau
Loss Function	Cross Entropy	Cross Entropy
Epochs Trained	26	332 (early stopping)
Best Epoch	26	$\sim$150
Best Val Accuracy	$\sim$87%	$\sim$29%
Infrastructure		
Framework	PyTorch	PyTorch
Hardware	Ada HPC	Ada HPC

Table 6: Hyperparameter comparison of CNN models

B Data

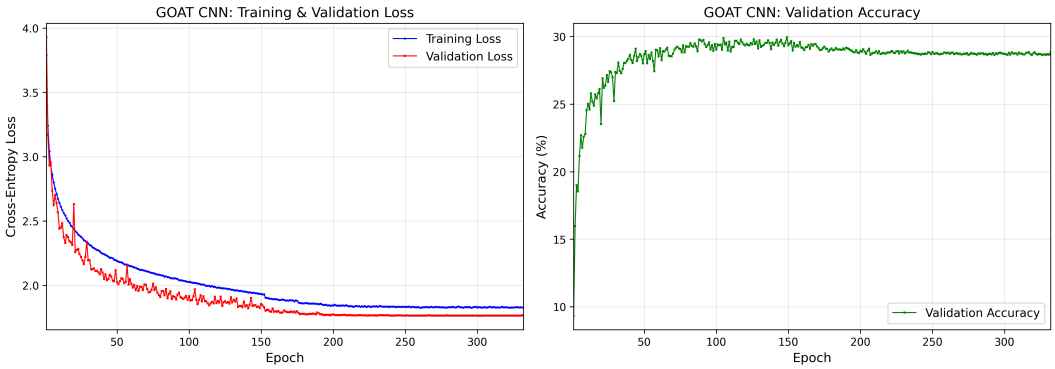

Figure 11: Loss Curve of GOAT CNN

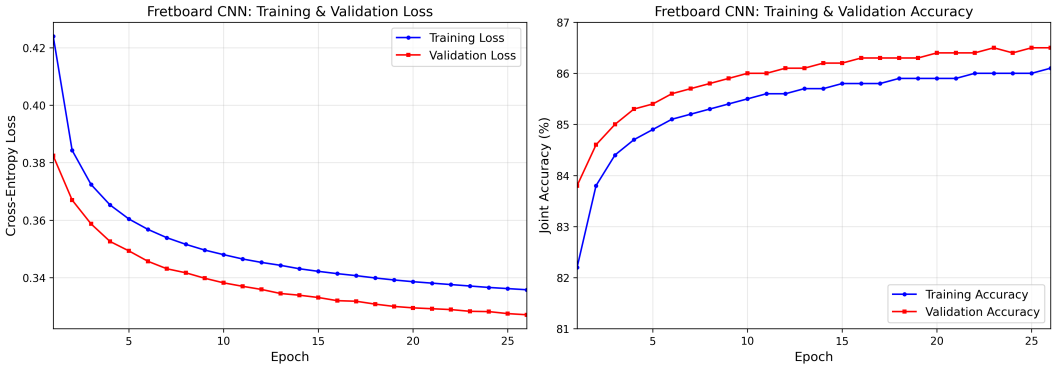

Figure 12: Loss Curve of Synth CNN

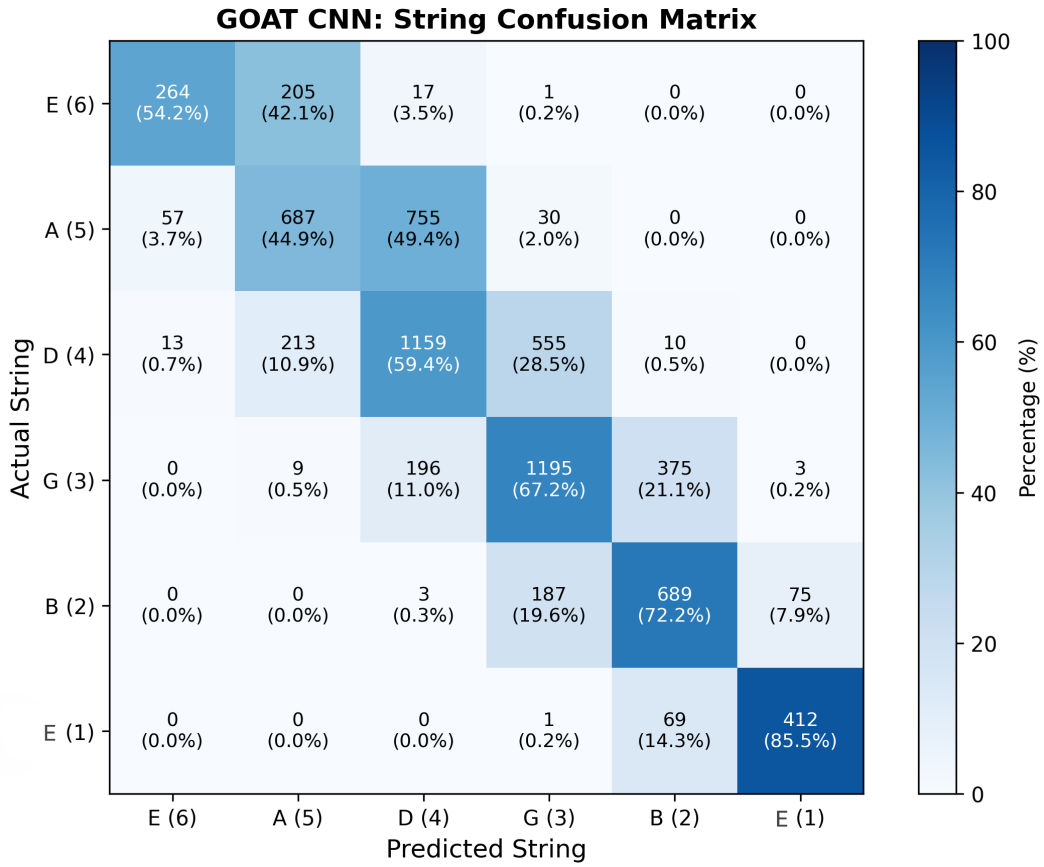

Figure 13: Confusion Matrix of GOAT CNN

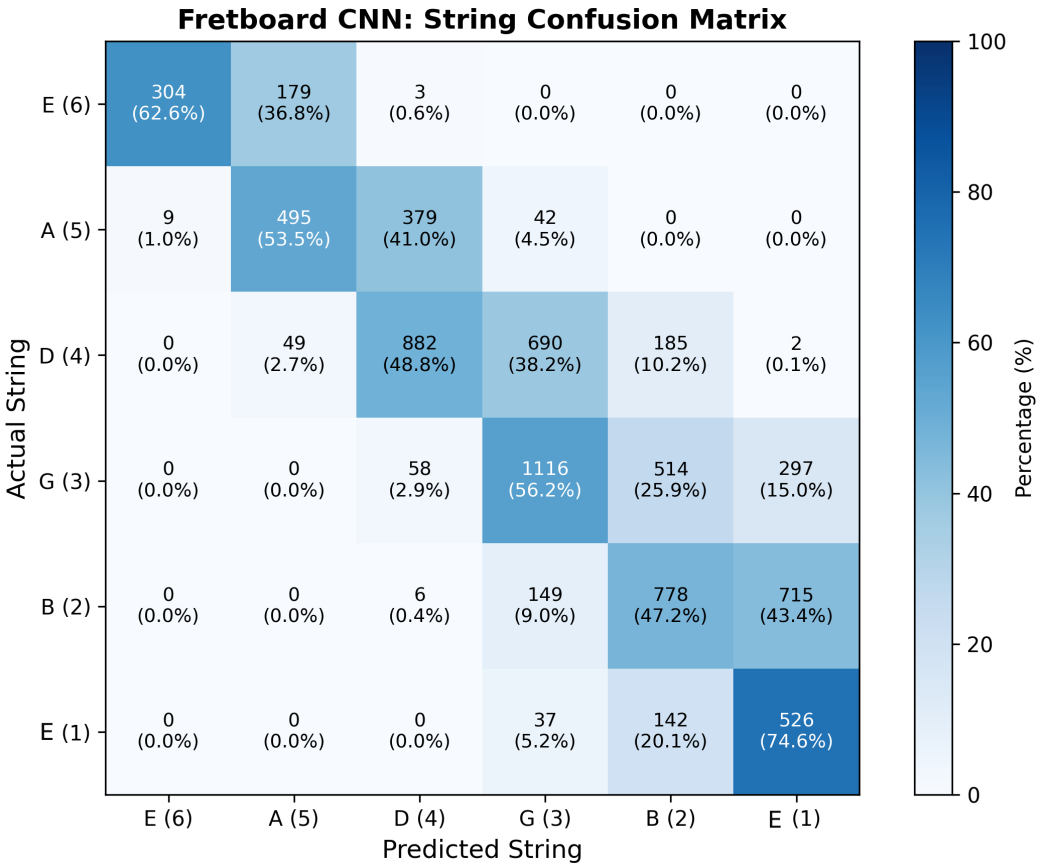

Figure 14: Confusion Matrix of Synth CNN

C Project Management

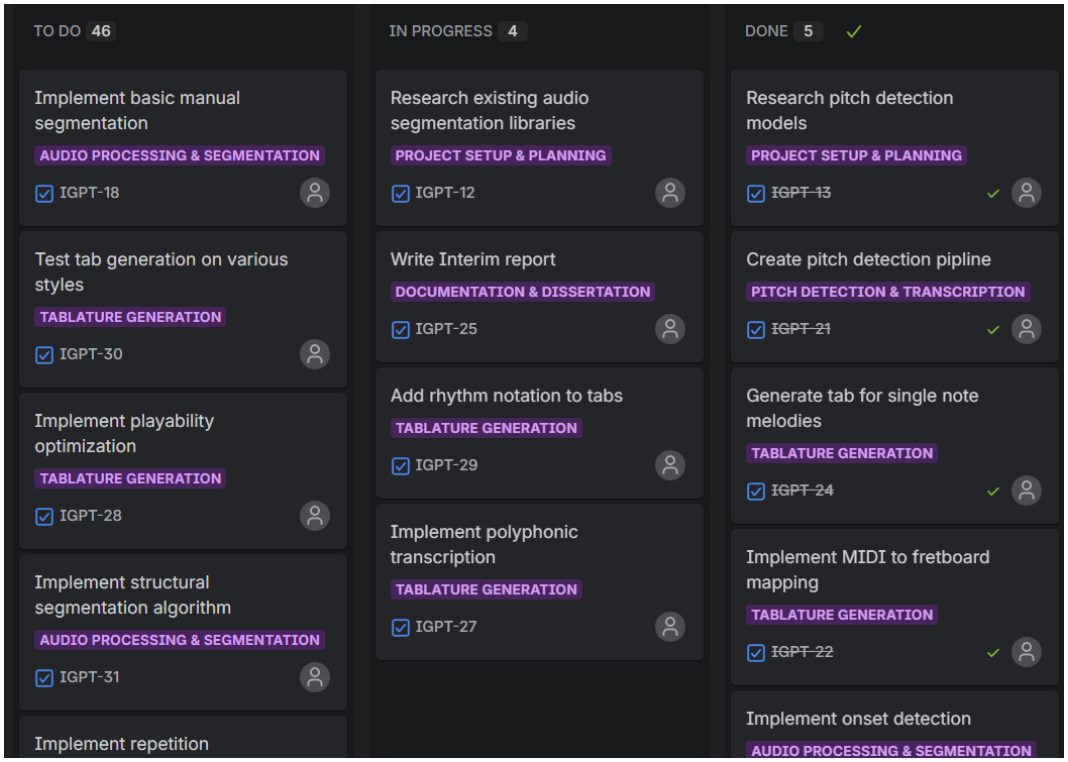

Figure 15: JIRA kanban board displaying tasks

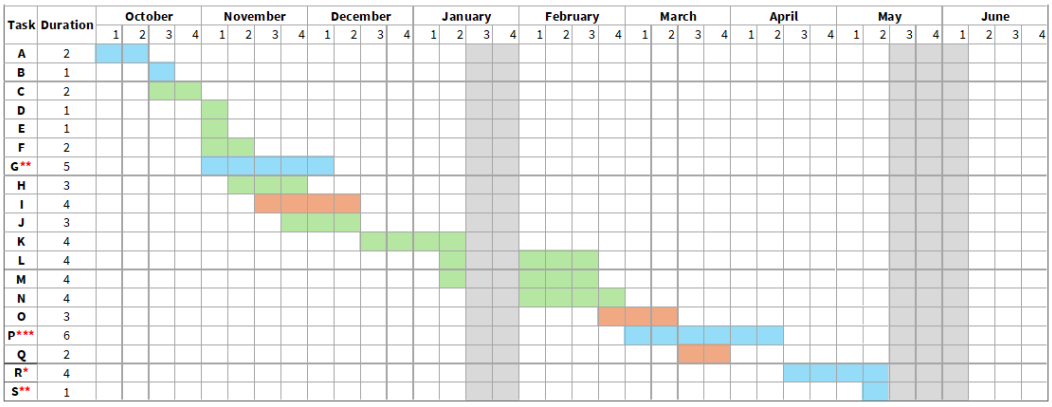

Figure 16: Gantt Chart Developed From Interim Report

ID	Task Description	Start	End	Duration
A	Complete Project Proposal	Oct W1	Oct W2	2 weeks
B	Complete Ethics and Data Management Plan	Oct W3	Oct W3	1 week
C	Foundation & Prototyping (Audio Loading, Basic Pitch)	Oct W3	Oct W4	2 weeks
D	Basic Tab Generation (Single Notes & Simple Melodies)	Oct W4	Oct W4	1 week
E	Multi Track Audio Separation	Nov W1	Nov W1	1 week
F	Improve Tab Generation (Chords, Polyphonic Audio)	Nov W1	Nov W2	2 weeks
G	Write & Submit Interim Report (10% of grade)	Nov W1	Dec W1	5 weeks
H	Implement Sequential Model (CNN Mapping)	Nov W2	Nov W4	3 weeks
I	Model Evaluation	Nov W3	Dec W2	4 weeks
J	Develop End-to-End Model	Nov W4	Dec W2	3 weeks
K	Guitar Technique Classification	Dec W3	Jan W2	4 weeks
L	User Interface Development	Jan W2	Mar W2	4 weeks
M	Automatic Audio Segmentation Implementation	Jan W2	Mar W1	4 weeks
N	Export Functionality for File Format	Feb W1	Mar W1	4 weeks
O	User Testing & Evaluation Study	Feb W4	Mar W3	3 weeks
P	Write Final Dissertation (75% of grade)	Mar W1	Apr W2	6 weeks
Q	Final Code Polish & Demo Video Creation	Mar W3	Mar W4	2 weeks
R	Prepare Presentation & Demo for Assessment	Apr W3	May W2	4 weeks
S	Final Presentation/Demonstration (15% of grade)	May W2	May W2	1 week

Figure 17: Updated project timeline

D Screenshots

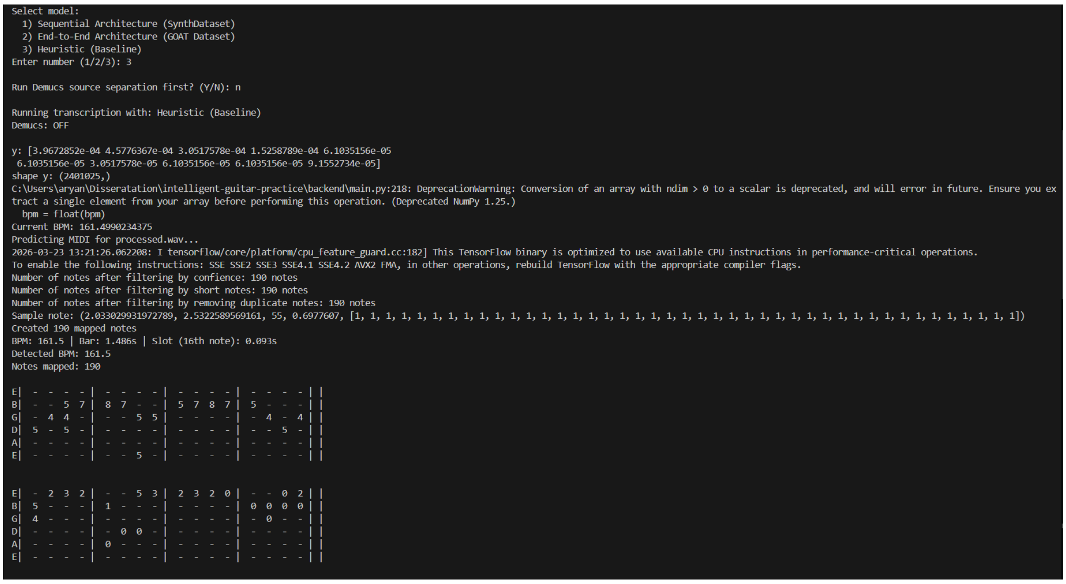

Figure 18: CLI version of the tab

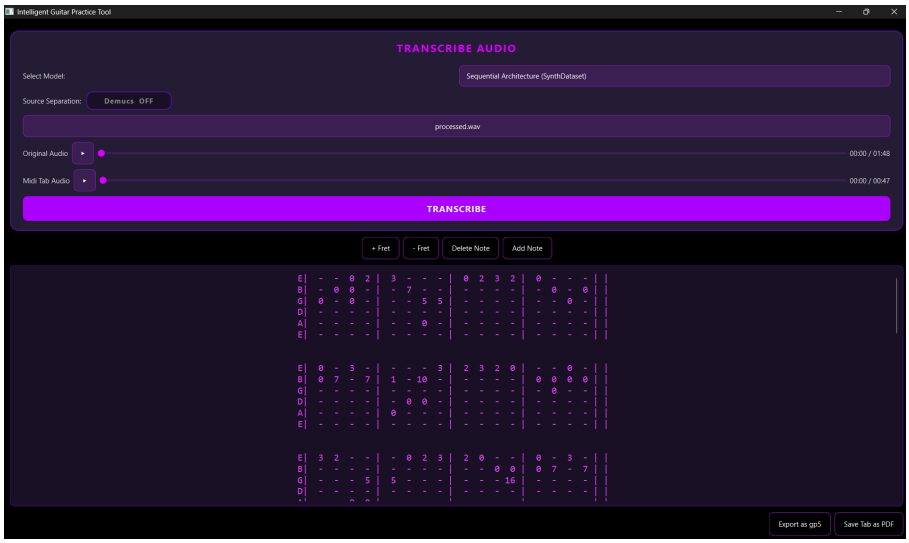

Figure 19: transcription tab page

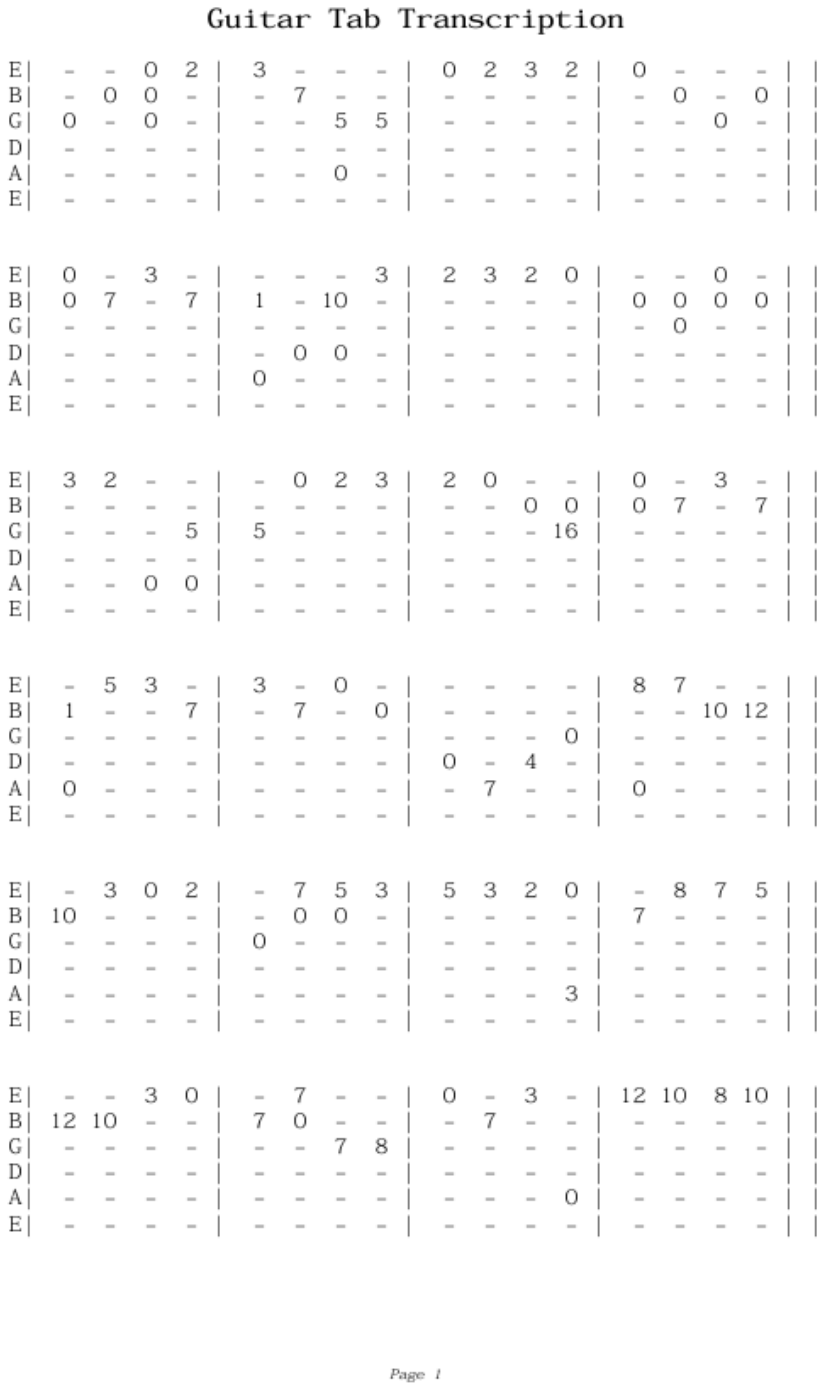

Figure 20: PDF Generated of The Tablature

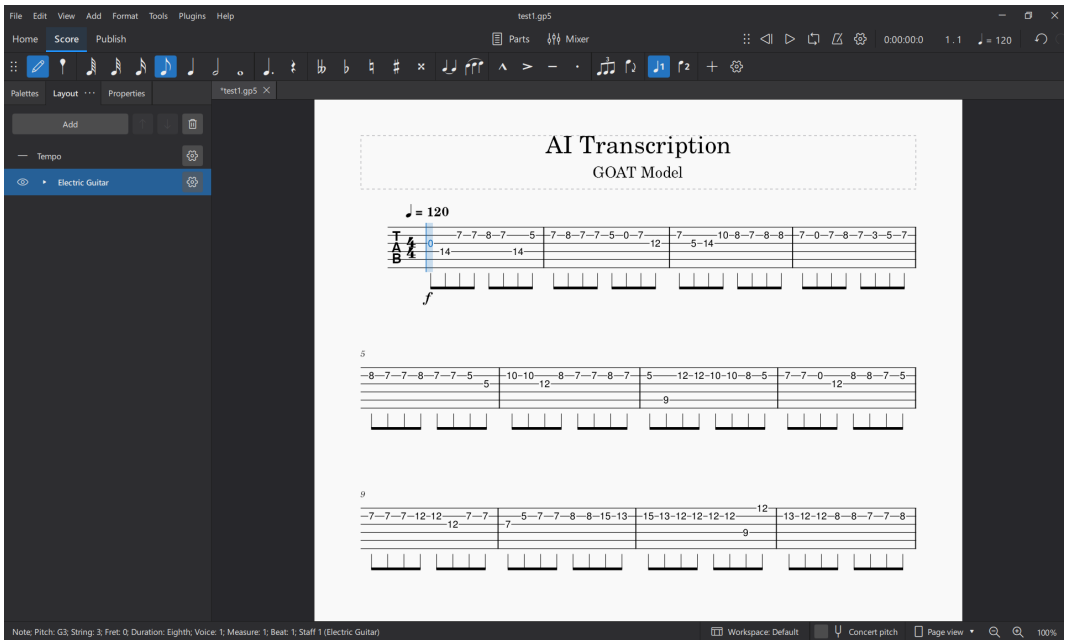

Figure 21: Tab generated by tool displayed in Muse Score 4

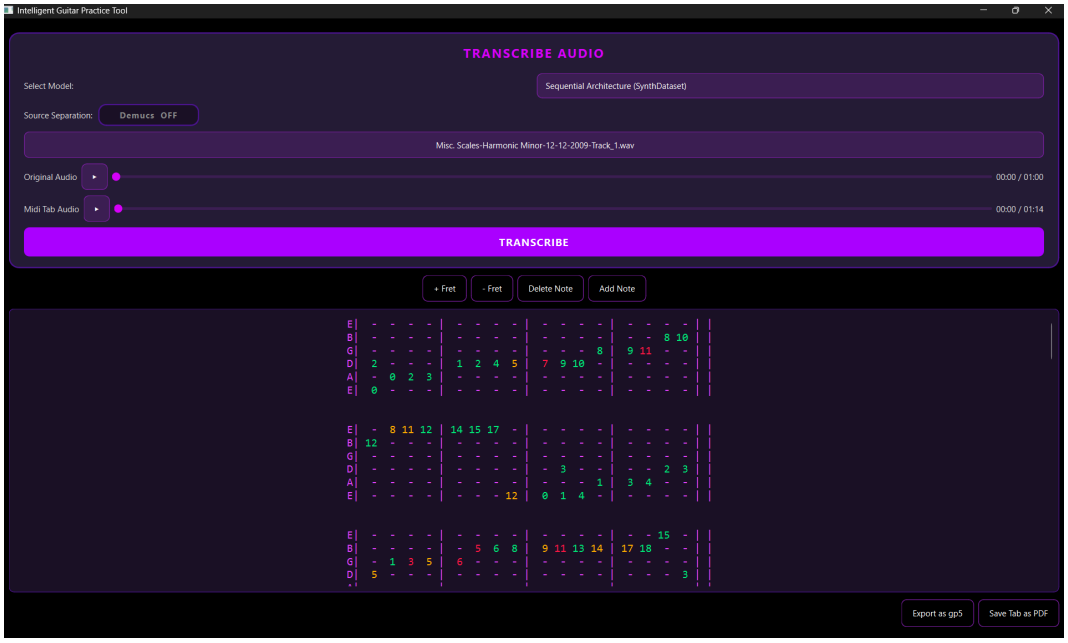

Figure 22: Monophonic Audio - tablature generated of the Harmonic Minor Scale

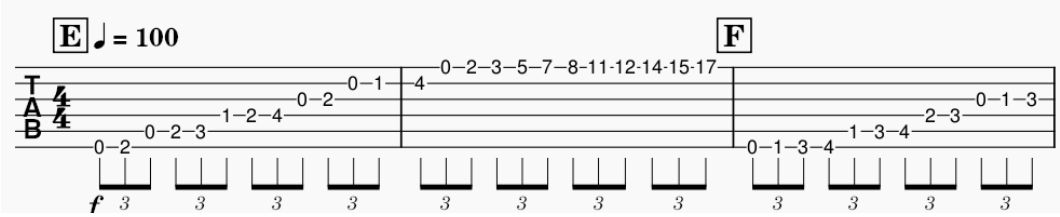

Figure 23: Monophonic Audio - Ground-truth tablature of the Harmonic Minor Scale

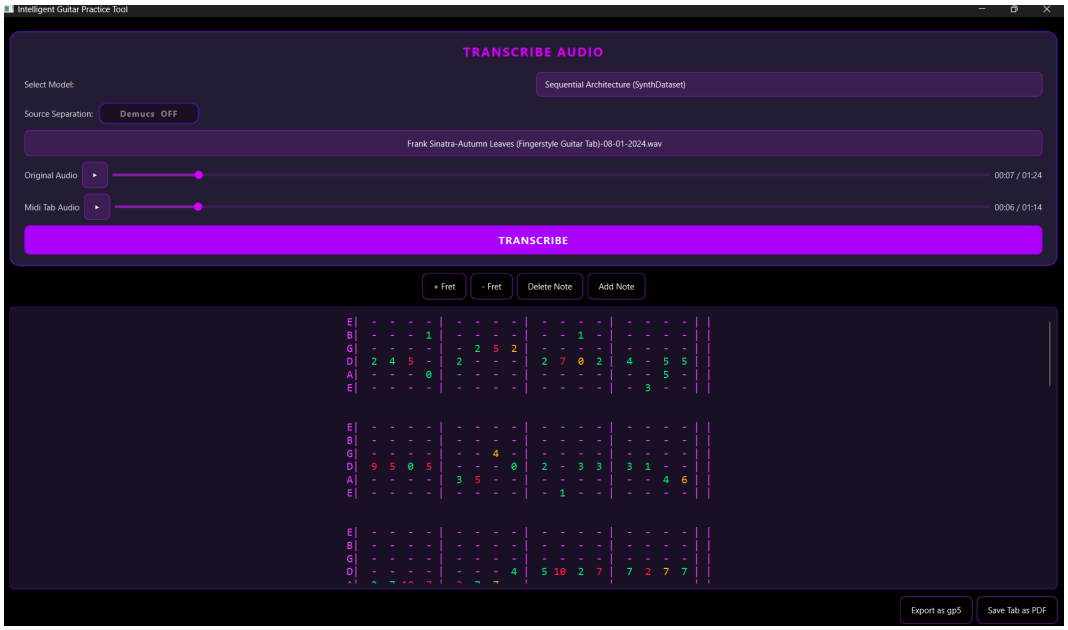

Figure 24: Polyphonic Audio - tablature generated of Frank Sinatra - Autumn Leaves

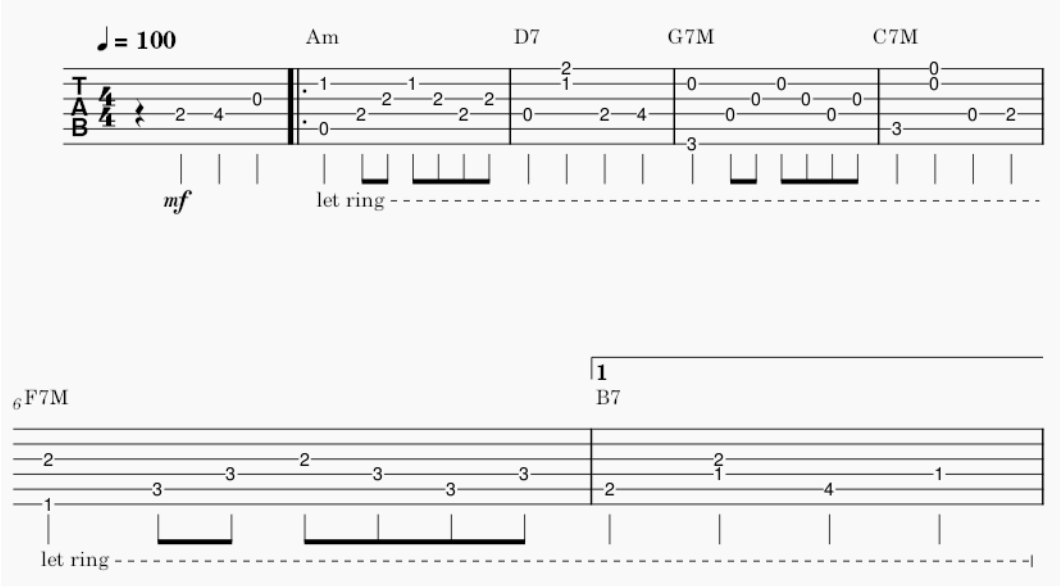

Figure 25: Polyphonic Audio - Ground-truth tablature of Frank Sinatra - Autumn Leaves

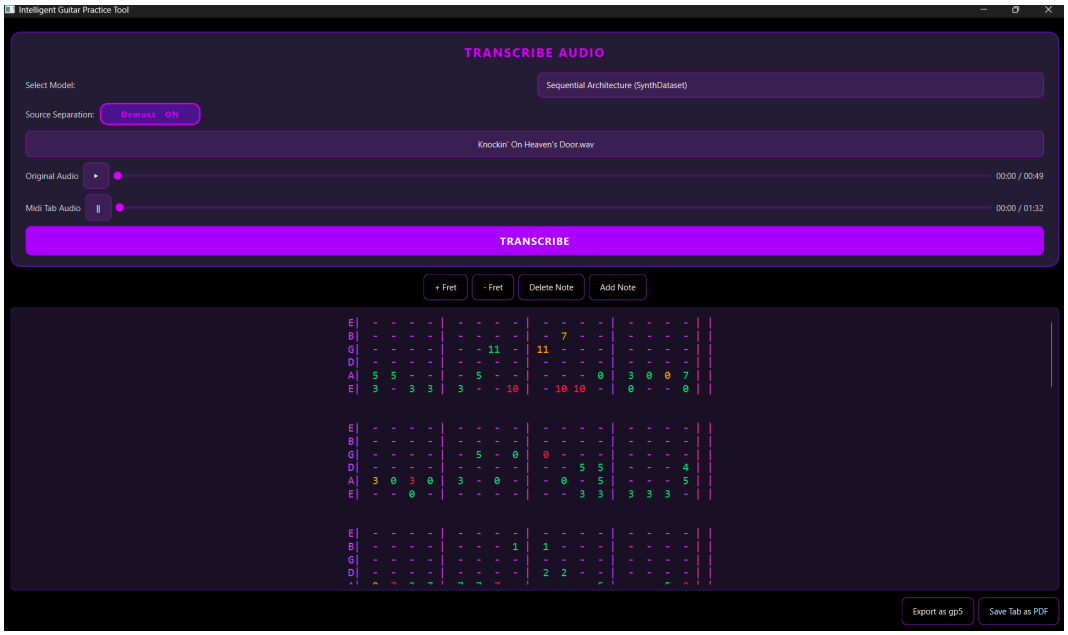

Figure 26: Multi-Channel Audio - tablature generated of Bob Dylan's - Knockin' On Heaven's Door

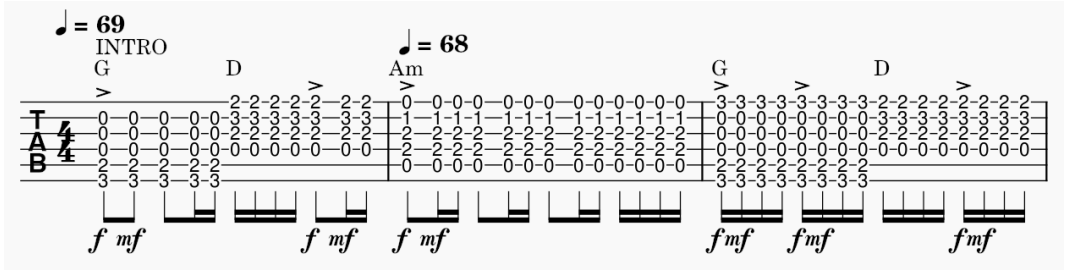

Figure 27: Monophonic Audio - Ground-truth tablature of Bob Dylan's - Knockin' on Heaven's Door

E Code Implementation

E.1 Fretboard Mapping — Heuristic Algorithm

The following function maps all detected MIDI notes to fretboard positions using the greedy nearest-neighbour heuristic described in Section 3.3.

```
def all_midi_notes(note_events, tuning_type):
    positions = []
    prev_pos = None
    for note in note_events:
```

```
        onset, offset, pitch_midi, velocity = note[0], note[1],
            note[2], note[3]
        pitch_midi = pitch_midi[0] if isinstance(pitch_midi,
            list) else pitch_midi
        candidates = map_note_to_fret(pitch_midi, tuning=
            tuning_type)
        if not candidates:
            continue
        best = best_position(prev_pos, candidates)
        positions.append((note, best))
        prev_pos = best
    return positions
```

The best_position function applies Manhattan Distance algorithm to the heuristic approach

```
def best_position(prev_pos, curr_pos):
    if not prev_pos:
        return min(curr_pos, key=lambda x: abs(x[1]-5)) #
            Middle of fretboard
    prev_string, prev_fret = prev_pos
    return min(curr_pos, key=lambda x: abs(x[0]-prev_string) +
        abs(x[1]-prev_fret))
```

E.2 CNN Model Definition

E.2.1 Fretboard CNN

The 1D CNN architecture for the sequential pipeline as described in Section 3.4.

```
def forward(self, input_pitch_data):
    embededed_x = self.pitch_embedding(input_pitch_data)
    transposed_x = embededed_x.transpose(1, 2)

    # convolution
    input_conv = self.conv1(transposed_x)
    hidden_conv = self.conv2(input_conv)
    output_conv = self.conv3(hidden_conv)

    # pooling
    flattened = output_conv.view(output_conv.size(0), -1)

    x = self.dropout(flattened)

```

```
    logits = self.total_class(x)

    return logits
```

E.2.2 GOAT CNN

The 2D CNN architecture for the end-to-end system as described in Section 5.4.

```
class GoatFretboardCNN(nn.Module):
    # 6 strings * 25 frets = 150 possible positions
    def __init__(self, harmonics=6, num_classes=150):
        super(GoatFretboardCNN, self).__init__()

        self.features = nn.Sequential(
            nn.Conv2d(harmonics, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1))
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(0.5),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

F Project Repository & Setup Guide

The source code for this project is available at:

`https://github.com/1102Aryan/intelligent-guitar-practice`

The following setup instructions are reproduced from the project's `README.md` file.

F.1 Installation

To run the desktop application on your device. Follow these instructions:

1. Visit the `https://github.com/1102Aryan/intelligent-guitar-practice/releases`.
2. Download the latest application: on Windows install .exe file, on MacOS install the zip file. 3. If zipped, extract it and then run.

To build from source code (for developers) warning: requires python 3.10 to match the needs of all libraries

1. Clone the repository: 'git clone https://github.com/1102Aryan/intelligent-guitar-practice.git'
2. Create a virtual environment: 'python -venv venv' 3. Activate the environment: 'venv\Scripts\activate'
4. Install the required libraries: 'pip install -r requirements.txt'
5. Run the application: `python .\run_app.py`